

Tong Xiao

Jingbo Zhu

Natural Language Processing

Neural Networks and Large Language Models

NATURAL LANGUAGE PROCESSING LAB

NORTHEASTERN UNIVERSITY

&

NIUTRANS RESEARCH

October 7, 2026

Copyright © 2021-2026 Tong Xiao and Jingbo Zhu

NATURAL LANGUAGE PROCESSING LAB, NORTHEASTERN UNIVERSITY
&
NIUTRANS RESEARCH

<https://github.com/NiuTrans/NLPBook>

<https://niutrans.github.io/NLPBook>

Licensed under the Creative Commons Attribution-NonCommercial 4.0 Unported License (the “License”). You may not use this file except in compliance with the License. You may obtain a copy of the License at <http://creativecommons.org/licenses/by-nc/4.0>. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

October 7, 2026

Chapter 5

Sequence-to-Sequence Models

天下万物之理，无独必有对。

According to the Principle of Heaven and Earth and all things, nothing exists in isolation but everything necessarily has its opposite.

– 《近思录》

Reflections on things at hand

朱熹/Xi Zhu (AD 1130-1200)

吕祖谦/Zuqian Lv (AD 1137-1181)

translated by [Chang \[1967\]](#)

In the world of language, things often come in pairs. If there is a question, there is an answer; if there is a text in Chinese, there is an English translation; if there is a sentence, there is a syntactic parse. Many NLP systems are designed to model the correspondence between such pairs, i.e., one element is taken as input and the other as output. These problems can be expressed in a form that we have encountered several times, as follows:

$$\hat{y} = \arg \max_y \Pr(y|x). \quad (5.1)$$

This chapter focuses on a particular family of problems where both x and y are sequences of words, known as **sequence-to-sequence** (or **seq2seq**) problems. Unlike classification problems, where the output $\hat{y}$ is selected from a fixed set of classes, sequence-to-sequence problems require the model to produce an output from an exponentially large output space. Obtaining $\hat{y}$ in this case is considerably more complex than in classification, as it requires more powerful models to represent $\Pr(y|x)$ and more efficient search algorithms to solve Eq. (5.1).

This chapter first introduces the well-known **encoder-decoder architecture** for sequence-to-sequence modeling, and then presents the attention mechanism as an important extension of this architecture. These models lay the foundation for several state-of-the-art approaches discussed in subsequent chapters. In addition, the chapter will discuss the search (decoding)

problem, which plays an important role in sequence generation and related tasks.

5.1 Sequence-to-Sequence Problems

We use machine translation as an illustrative example throughout this chapter, because it is now one of the most popular sequence-to-sequence tasks. We use $\mathbf{x} = x_1 \dots x_m$ to denote a sequence of words in one language (call it a **source-side sequence** or **source sequence**), and use $\mathbf{y} = y_1 \dots y_n$ to denote a sequence of words in another language (call it a **target-side sequence** or **target sequence**). Using this notation, Eq. (5.1) can be written as:

$$\begin{aligned}\hat{\mathbf{y}} &= \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x}) \\ &= \arg\max_{y_1 \dots y_n} \Pr(y_1 \dots y_n | x_1 \dots x_m).\end{aligned}\tag{5.2}$$

As discussed in Chapter 1 and in [Brown et al., 1993], this formulation implies three fundamental issues.

- **Modeling.** First, we need to define the form of $\Pr(\mathbf{y}|\mathbf{x})$. In this chapter we show that $\Pr(\mathbf{y}|\mathbf{x})$ can be modeled using a single neural network based on the encoder-decoder architecture and the attention mechanism. Note that sometimes we just need a model for discriminating “good” from “bad” target sequences. In this case, it is not necessary to require the model to be probabilistic, and we can take a discriminant function instead.
- **Training.** Then, we need to learn the parameters of the model $\Pr(\mathbf{y}|\mathbf{x})$ given some training data. As $\Pr(\mathbf{y}|\mathbf{x})$ is implemented as a neural network, we can train it in a standard fashion: we optimize a loss function by gradient descent. See Chapter 3 for common approaches to training neural networks. We will also discuss techniques that are tailored for specific tasks in this and the following chapters.
- **Search (or Decoding).** Once we have learned a model, we will obtain $\hat{\mathbf{y}}$ by searching for a target sequence that maximizes $\Pr(\mathbf{y}|\mathbf{x})$. This poses a computational challenge because the number of candidate sequences grows with the maximum sequence length and the size of the vocabulary. In Section 5.4, we will discuss efficient and effective search methods for sequence-to-sequence problems, particularly for machine translation.

Many NLP problems that fit the form of Eq. (5.2) can be formulated as sequence-to-sequence problems. Table 5.1 shows common examples of sequence-to-sequence problems taken from the literature. When the target side is text, such problems can broadly be categorized as **text generation** problems, although a general text generation system does not require the source-side to be sequential. In addition to language and speech processing, sequence-to-sequence problems can be generalized to cases where the input and/or output of a system are not naturally sequential. For example, **image-to-text generation** (or **image captioning**) and **text-to-image generation** systems both involve dealing with images that are typically represented as 2D data. By representing images as sequences in some way (such as sequences of patches), sequence-to-sequence models are directly applicable to these tasks.

Task	Source	Target
Machine Translation	Text in One Language	Text in Another Language
Question Answering	Question	Answer
Dialogue Systems	Dialogue Context	Response
Summarization	Text	Summary
Text Simplification	Text	Simplified Text
Text Style Transfer	Text in One Style	Text in Another Style
Grammar Correction	Text with Errors	Corrected Text
Speech Recognition	Speech	Text
Speech Synthesis	Text	Speech
Speech Translation	Speech in One Language	Speech or Text in Another Language

Table 5.1: Examples of sequence-to-sequence problems.

Historically, most systems in these tasks were developed somewhat independently, resulting in different architectures, features, and training methods for different tasks. However, as shown in this chapter, when we represent these models as neural networks and train them in an end-to-end fashion, there appears to be a “universal” paradigm for all these problems. This is a big change for the AI community because many research fields come together and systems can be shared across them. We can gain some insight into the common nature of a broad variety of problems, though there are many task-specific considerations in practice. In the following sections, we will discuss some of the common threads among sequence-to-sequence models.

5.2 The Encoder-Decoder Architecture

In this section we discuss the encoder-decoder architecture and a simple neural machine translation model built upon this architecture.

5.2.1 Encoding and Decoding

From a supervised learning viewpoint, we would ideally like to learn a model from a number of sequence pairs such that any source-side sequence can be mapped to the corresponding target-side sequence. However, learning mappings between sequences of discrete variables often requires handling high-dimensional data, which inevitably suffers from the curse of dimensionality.

One approach to learning such a mapping is to divide the problem into simpler sub-problems. We assume that there is a low-dimensional representation shared by $\mathbf{x}$ and $\mathbf{y}$, denoted by $\mathbf{H}$. Then, the mapping $\mathbf{x} \rightarrow \mathbf{y}$ can be achieved by mapping $\mathbf{x}$ to $\mathbf{H}$ and then to $\mathbf{y}$. Formally, given a source-side sequence $\mathbf{x}$, we map it to the representation $\mathbf{H}$ using an **encoding**

system (call it an encoder)

$$\mathbf{H} = \text{Encode}(\mathbf{x}). \quad (5.3)$$

Then, we map $\mathbf{H}$ to the target-side sequence $\mathbf{y}$ by using a **decoding system** (call it a decoder)¹

$$\mathbf{y} = \text{Decode}(\mathbf{H}). \quad (5.4)$$

This architecture, also known as the encoder-decoder architecture, is widely used in modern sequence-to-sequence systems (see Figure 5.1 for an illustration). It is easy to see that the form of Eq. (5.3) is the same as that of the sequence models mentioned in Chapter 4, and so there are many encoding models to choose from, such as bi-directional LSTM. The goal of the decoder is to produce a “best” target-side sequence given the representation of the source-side sequence. Like classification models, the prediction is made by first producing a distribution over all possible sequences, and then selecting the one with the maximum probability. As such, we can re-define $\text{Decode}(\cdot)$ as a probability function

$$\begin{aligned} \Pr(\cdot|\mathbf{H}) &= \text{Decode}(\mathbf{H}) \\ &= \text{Decode}(\text{Encode}(\mathbf{x})). \end{aligned} \quad (5.5)$$

In other words, given a target-side sequence $\mathbf{y}$ and a source-side sequence $\mathbf{x}$, the decoder assigns a probability to $\mathbf{y}$

$$\Pr(\mathbf{y}|\mathbf{x}) = \Pr(\mathbf{y}|\mathbf{H}). \quad (5.6)$$

Then, the optimal sequence $\hat{\mathbf{y}}$ is obtained by performing $\arg \max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x})$ as in Eq. (5.2). In many systems based on the encoder-decoder architecture, both $\text{Encode}(\cdot)$ and $\text{Decode}(\cdot)$ are models constructed from neural networks. Thus, we can treat the sequence-to-sequence model as a single neural network and train it as usual, provided the entire model is some combination of $\text{Encode}(\cdot)$ and $\text{Decode}(\cdot)$.

To apply the encoder-decoder architecture to a real-world task, we need to make a number of design choices, such as the forms of $\mathbf{H}$, $\text{Encode}(\cdot)$ and $\text{Decode}(\cdot)$. As a very simple example, consider the task of reconstructing an input word. We can define $\text{Encode}(\cdot)$ as a

¹It is important to distinguish between the concept of *decoding* (or *decoder*) used in conventional sequence-to-sequence systems and that used in the encoder-decoder architecture. The two are often confused, though they are different in nature. In many machine translation or speech recognition systems, *decoding* has the same meaning as *translation* or *transcription*, that is, we recover the optimal $\mathbf{y}$ from $\mathbf{x}$. As pointed out in Eq. (5.2), this process involves a search over all candidate $\mathbf{y}$. Therefore, the conventional use of decoding in these systems is to refer to a search process (i.e., the $\arg \max$ operation in Eq. (5.2)) [Koehn, 2010]. By contrast, in the encoder-decoder architecture *decoding* means a process of recovering the target-side sequence $\mathbf{y}$ from the intermediate representation $\mathbf{H}$. It is all about modeling rather than searching. It is also worth noting that, while the term *decoding* (or *decoder*) is used in different ways, it can be thought of as a process of mapping an encoded message back to the original message in a communication system as defined in information theory [Shannon, 1948]. In this sense, the *decoding* processes in these systems do the same thing as the word sounds like: convert something to its original form.

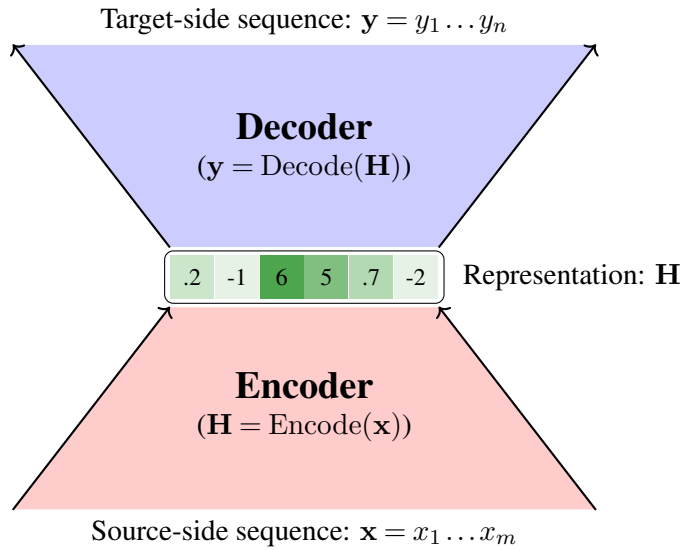


Figure 5.1: The encoder-decoder architecture. In the case of sequence-to-sequence problems, it transforms a source-side sequence $\mathbf{x} = x_1 \dots x_m$ to a target-side sequence $\mathbf{y} = y_1 \dots y_n$. This procedure involves two steps: $\mathbf{x}$ is first encoded as a representation $\mathbf{H}$, and this representation is then decoded to $\mathbf{y}$.

feed-forward neural network that takes a word (in one-hot representation) and outputs a word vector. In this way, $\mathbf{H}$ is a distributed representation of the word. Then, we define $\text{Decode}(\cdot)$ as another feed-forward neural network that takes the word vector and generates a distribution over the vocabulary. For training, we wish to learn a system that assigns the largest probability to the input word. As discussed in Chapter 2, we can call this an auto-encoder which is a special instance of the encoder-decoder architecture.

5.2.2 Example: Neural Machine Translation

Next, we illustrate the application of the encoder-decoder architecture using a working example: **neural machine translation** (NMT). We consider a well-known NMT model that employs RNNs or their variants for both the encoder and the decoder [Cho et al., 2014; Sutskever et al., 2014]. The encoder of this NMT model is a standard RNN-based sequence model. Since the RNN-based sequence model has been discussed in detail in Chapter 4, we provide only a brief review here. Suppose that the source-side vocabulary is V_x and each source-side word x_j is represented as a one-hot vector in $\mathbb{R}^{|V_x|}$. Then, x_j is transformed into an h_s -dimensional vector (or word embedding)

$$\mathbf{x}_j^e = \text{Embed}_s(x_j), \quad (5.7)$$

where $\text{Embed}_s(\cdot)$ is the word embedding function. More details about word embedding models can be found in Chapter 3.

The RNN model takes the sequence of word embeddings $\mathbf{x}_1^e \dots \mathbf{x}_m^e$ and produces a sequence

of RNN state vectors $\mathbf{h}_1 \dots \mathbf{h}_m$. An RNN state vector $\mathbf{h}_j \in \mathbb{R}^{d_h}$ is defined as

$$\mathbf{h}_j = \text{RNN}(\mathbf{h}_{j-1}, \mathbf{x}_j^e). \quad (5.8)$$

Here, $\text{RNN}(\cdot)$ is an RNN unit that summarizes the information up to position j by combining the previous state $\mathbf{h}_{j-1}$ and the current input $\mathbf{x}_j^e$. Then, the final state $\mathbf{h}_m$ can be treated as a compressed representation of the input sequence $x_1 \dots x_m$, and we use $\mathbf{h}_m$ as the output of the encoder, written as

$$\mathbf{h}_m = \text{Encode}(x_1 \dots x_m). \quad (5.9)$$

Figure 5.2 (a-b) illustrates the encoding process. Note that the model described above involves only a single-layer RNN. In practical systems, this framework can be easily extended to include multiple layers and more powerful recurrent units (such as LSTM units).

The decoder of the NMT model is a standard RNN-based language model, that is, it predicts the next word y_{i+1} given all previous words $y_1 \dots y_i$. To incorporate the source-side information into the translation, a straightforward method is to treat $\mathbf{h}_m$ as the initial state of the target-side RNN. Let $\mathbf{y}_0^e \in \mathbb{R}^{d_s}$ be the word vector of the start symbol $\langle \text{SOS} \rangle$ (denoted by y_0). The corresponding initial RNN state is given by

$$\mathbf{s}_0 = \text{RNN}(\mathbf{h}_m, \mathbf{y}_0^e). \quad (5.10)$$

Here, $\text{RNN}(\cdot)$ has the same form as the recurrent unit used in the encoder, but with different parameters.

For $i > 0$, the state vector $\mathbf{s}_i \in \mathbb{R}^{d_s}$ is defined as

$$\mathbf{s}_i = \text{RNN}(\mathbf{s}_{i-1}, \mathbf{y}_i^e). \quad (5.11)$$

Then, $\mathbf{s}_i$ is fed into a Softmax layer to produce a distribution over the target-side vocabulary V_y . The output of the Softmax layer is given by

$$\begin{aligned} \Pr(\cdot | y_1 \dots y_i, x_1 \dots x_m) &= \Pr(\cdot | \mathbf{s}_i) \\ &= \text{Softmax}(\mathbf{s}_i \mathbf{U}_y + \mathbf{b}_y), \end{aligned} \quad (5.12)$$

where $\mathbf{U}_y \in \mathbb{R}^{d_s \times |V_y|}$ and $\mathbf{b}_y \in \mathbb{R}^{|V_y|}$ are the parameters of the Softmax layer. $\Pr(y_{i+1} | y_1 \dots y_i, x_1 \dots x_m)$ can be seen as the probability of predicting word y_{i+1} conditioned on both the translated words $y_1 \dots y_i$ and the source-side sequence $x_1 \dots x_m$. See Figure 5.2 (c-d) for an illustration of the word predictions of a decoder.

Based on this word prediction model, the joint probability frequently used in NMT literature

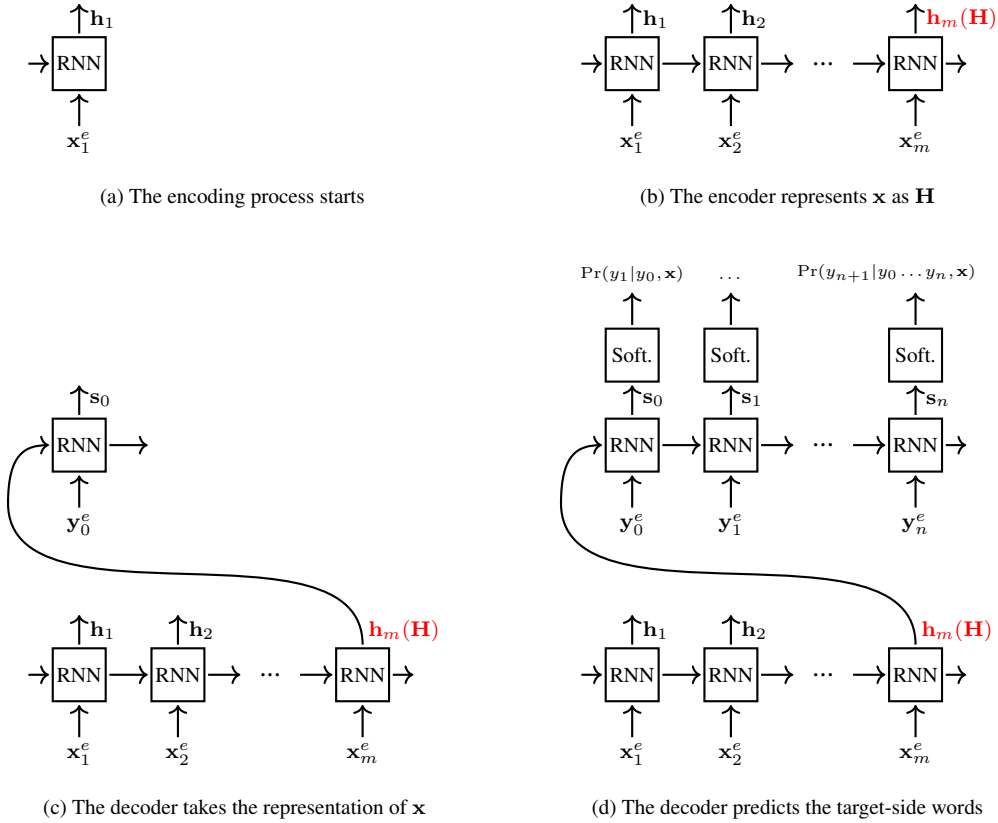


Figure 5.2: The encoding and decoding steps for an RNN-based NMT system. The encoder is a standard RNN. The encoding process starts with the first source-side word and concludes with the last source-side word. The final state of the RNN is taken as the representation of the entire source-side sequence (i.e., $\mathbf{H} = \mathbf{h}_m$). The decoder is another RNN. In the first step, it receives $\mathbf{H}$ from the encoder. After representing $(\mathbf{y}_0^e \dots \mathbf{y}_i^e, \mathbf{H})$ as $\mathbf{s}_i$ at position i , a Softmax layer is applied to predict the next word y_{i+1} .

is formulated as follows:

$$\begin{aligned}
 \Pr(\mathbf{y}|\mathbf{x}) &= \Pr(y_0 y_1 \dots y_n | x_1 \dots x_m) \\
 &= \Pr(y_0 | x_1 \dots x_m) \Pr(y_1 \dots y_n | y_0, x_1 \dots x_m) \\
 &= \prod_{i=0}^{n-1} \Pr(y_{i+1} | y_0 \dots y_i, x_1 \dots x_m).
 \end{aligned} \tag{5.13}$$

Sometimes, this equation is also written in an equivalent form:

$$\Pr(\mathbf{y}|\mathbf{x}) = \prod_{i=1}^n \Pr(y_i | y_0 \dots y_{i-1}, x_1 \dots x_m). \tag{5.14}$$

Here, we assume that $\mathbf{y}$ always starts with y_0 (i.e., $\langle \text{SOS} \rangle$), such that $\Pr(y_0 | x_1 \dots x_m) = 1$.

In many practical systems, it is also common to assume that $\mathbf{y}$ ends with a special symbol $\langle \text{EOS} \rangle$. Therefore, we can modify this equation to include $\langle \text{SOS} \rangle$ and $\langle \text{EOS} \rangle$ on both the source and target sides, as follows:

$$\begin{aligned}
 \Pr(y_0 \mathbf{y} y_{n+1} | x_0 \mathbf{x} x_{m+1}) &= \Pr(y_0 y_1 \dots y_n y_{n+1} | x_0 x_1 \dots x_m x_{m+1}) \\
 &= \Pr(y_0 | x_0 \dots x_{m+1}) \cdot \\
 &\quad \Pr(y_1 \dots y_n y_{n+1} | y_0, x_0 \dots x_{m+1}) \\
 &= \prod_{i=0}^n \Pr(y_{i+1} | y_0 \dots y_i, x_0 \dots x_{m+1}), \tag{5.15}
 \end{aligned}$$

where $x_0 = y_0 = \langle \text{SOS} \rangle$, $x_{m+1} = y_{n+1} = \langle \text{EOS} \rangle$, and $\Pr(y_0 | x_0 x_1 \dots x_m x_{m+1}) = 1$.

Since $\Pr(\mathbf{y} | \mathbf{x})$ can be implemented as a neural network, training this model is straightforward. As described in Chapter 4, RNN-based language models are trained using cross-entropy loss and gradient descent. NMT uses this same method for training model parameters. Once we obtain the optimized model, we can use it to translate new sentences. Finding the best translation for any given source-side sentence is a standard search problem, which we will discuss in Section 5.4.

5.3 The Attention Mechanism

The NMT model discussed in the previous section is based on a fixed-length representation of the source-side sequence. While this model is easy to implement, in many practical applications it is often inadequate, as a fixed-length vector is insufficient to represent a variable-length sequence, especially when the sequence is long. We therefore need some mechanism to couple the encoder and the decoder in a fine-grained manner. In this section we discuss the attention mechanism by which the model can learn, for each word in the target-side sequence, an adaptive representation that focuses more on important parts of the source-side sequence.

This idea is closely related to models of attention in psychology because translation is itself a cognitive process [Sternberg, 1996; Neisser, 2014]. The key idea behind this type of model is intuitive: attention is generally concentrated on specific parts of the input. This intuition forms the basis of many state-of-the-art sequence-to-sequence models, and the attention mechanism has been the de facto standard in sequence-to-sequence modeling.

5.3.1 A Basic Model

Recall that in the NMT model of the previous section, the encoder represents a source-side word sequence as $\mathbf{h}_1, \dots, \mathbf{h}_m$, and the decoder represents a target-side word sequence as $\mathbf{s}_1, \dots, \mathbf{s}_n$. The attention mechanism addresses the question of how a representation can be learned from $\mathbf{h}_1, \dots, \mathbf{h}_m$ so that this representation can encode the source-side sequence well for a given query state $\mathbf{s}_i$ ². From an information processing perspective, so long as we ignore

²Following the convention in machine translation [Brown et al., 1993], we use j to represent a position in the source-side sequence, and use i to represent a position in the target-side sequence.

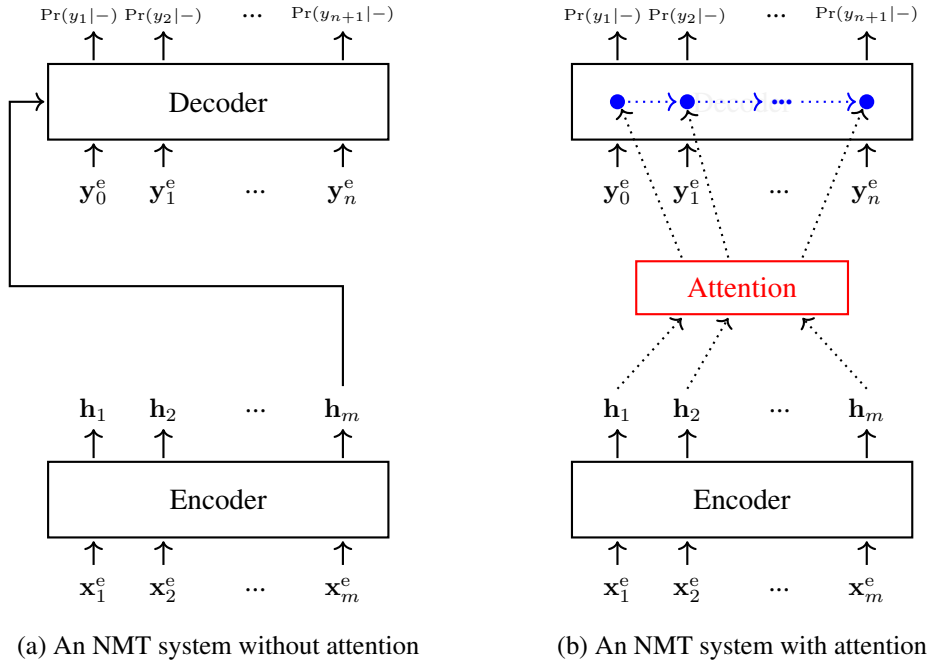


Figure 5.3: NMT architectures without (left) and with (right) the attention model. When the attention model is not involved, a fixed-length representation is considered for generating the entire target-side sequence. By contrast, when the attention model is involved, a new representation is computed specifically for each target-side state so that the decoder can learn to concentrate on different parts of the source-side sequence for predicting a target-side word.

the meanings of $\mathbf{h}_1, \dots, \mathbf{h}_m$ and $\mathbf{s}_i$ in NMT, attention can be thought of as a generic mechanism for processing the input information $\mathbf{h}_1, \dots, \mathbf{h}_m$ by considering how each $\mathbf{h}_j$ is related to the query state $\mathbf{s}_i$. Figure 5.3 compares NMT architectures with and without the attention mechanism.

More formally, an attention model produces a weighted combination of $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ in the form

$$\mathbf{c}_i = \sum_{j=1}^m \alpha_{i,j} \cdot \mathbf{h}_j, \quad (5.16)$$

where $\alpha_{i,j}$ is the **attention weight** that describes how much the model should rely on $\mathbf{h}_j$ when computing $\mathbf{c}_i$ for $\mathbf{s}_i$. Sometimes $\mathbf{c}_i$ is also called a **context vector**.

A common approach to computing attention weights is to normalize **alignment scores** in the following form

$$\alpha_{i,j} = \frac{\exp(a(\mathbf{s}_i, \mathbf{h}_j))}{\sum_{j'=1}^m \exp(a(\mathbf{s}_i, \mathbf{h}_{j'}))}. \quad (5.17)$$

Here the alignment score $a(\mathbf{s}_i, \mathbf{h}_j)$ measures how strongly $\mathbf{h}_j$ is related to $\mathbf{s}_i$. In general,

$a(\mathbf{s}_i, \mathbf{h}_j)$ can be defined in several different ways [Graves et al., 2014; Bahdanau et al., 2014; Luong et al., 2015]. A comprehensive list of these functions can be found in survey papers on this subject [Chaudhari et al., 2021]. Here we introduce some of the common ones.

- **Dot-product Attention.** One of the simplest methods is to measure the similarity between $\mathbf{h}_j$ and $\mathbf{s}_i$. Thus, we can calculate the dot-product of the two vectors, as follows

$$\begin{aligned} a(\mathbf{s}_i, \mathbf{h}_j) &= \mathbf{s}_i \mathbf{h}_j^\top \\ &= \sum_{k=1}^{d_h} s_i(k) \cdot h_j(k). \end{aligned} \quad (5.18)$$

A variant of this model, called **scaled dot-product attention**, applies a scaling factor of $\frac{1}{\beta}$ to the right-hand side of Eq. (5.18), as follows

$$a(\mathbf{s}_i, \mathbf{h}_j) = \frac{\mathbf{s}_i \mathbf{h}_j^\top}{\beta}, \quad (5.19)$$

where β is a hyperparameter. We will see an example of this model later in this section.

- **Cosine Attention.** Another commonly used similarity measure in vector algebra is the cosine of the angle between two vectors, given by

$$\begin{aligned} a(\mathbf{s}_i, \mathbf{h}_j) &= \cos(\mathbf{s}_i, \mathbf{h}_j) \\ &= \frac{\mathbf{s}_i \mathbf{h}_j^\top}{\|\mathbf{s}_i\|_2 \cdot \|\mathbf{h}_j\|_2}, \end{aligned} \quad (5.20)$$

where $\|\mathbf{a}\|_2 = (\mathbf{a} \cdot \mathbf{a})^{\frac{1}{2}}$ is the Euclidean norm of the vector $\mathbf{a}$.

- **Weighted Dot-product Attention.** This attention model involves a linear mapping of the input vectors before performing the dot-product operation, given by

$$a(\mathbf{s}_i, \mathbf{h}_j) = \mathbf{s}_i \mathbf{W}_a \mathbf{h}_j^\top, \quad (5.21)$$

where $\mathbf{W}_a \in \mathbb{R}^{d_h \times d_h}$ is the parameter matrix of the linear mapping. Both this approach and the dot-product attention approach are also called **multiplicative attention** [Ruder, 2017].

- **Additive Attention.** In additive attention, the entries of the two vectors are summed in some way. A widely-used form is given by Bahdanau et al. [2014]

$$a(\mathbf{s}_i, \mathbf{h}_j) = \mathbf{v}_a^\top \text{TanH}(\mathbf{s}_i \mathbf{W}_s + \mathbf{h}_j \mathbf{W}_h), \quad (5.22)$$

where $\mathbf{W}_h, \mathbf{W}_s \in \mathbb{R}^{d_h \times d_a}$ and $\mathbf{v}_a \in \mathbb{R}^{d_a}$ are parameters. $\text{TanH}(\mathbf{s}_i \mathbf{W}_s + \mathbf{h}_j \mathbf{W}_h)$ produces a d_a -dimensional vector where each entry is a transformed weighted sum of the entries of $\mathbf{h}_j$ and $\mathbf{s}_i$. It is followed by a dot-product with another weight vector $\mathbf{v}_a$.

Now let us return to Eqs. (5.16-5.17) and rethink the role of attention weights. Eq. (5.17)

informally defines a “distribution” over $\mathbf{h}_1 \dots \mathbf{h}_m$, written as

$$\Pr(\mathbf{h}_j | \mathbf{s}_i) = \alpha_{i,j}. \quad (5.23)$$

If we consider $\mathbf{h}$ as a random variable that takes a value from $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$, then $\alpha_{i,j}$ can be thought of as the probability of $\mathbf{h} = \mathbf{h}_j$, conditioned on $\mathbf{s}_i$, and Eq. (5.16) can be rewritten as

$$\begin{aligned} \mathbf{c}_i &= \sum_{j=1}^m \Pr(\mathbf{h}_j | \mathbf{s}_i) \cdot \mathbf{h}_j \\ &= \mathbb{E}_{\mathbf{h} \sim \Pr(\mathbf{h} | \mathbf{s}_i)}(\mathbf{h}). \end{aligned} \quad (5.24)$$

In other words, $\mathbf{c}_i$ can be viewed as an **expected representation** of the source-side sequence given the target-side state $\mathbf{s}_i$, that is, the expectation of $\mathbf{h}$ over the set $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ under the distribution $\Pr(\mathbf{h}_j | \mathbf{s}_i)$. This provides a general framework for describing the way the decoder receives the information from the encoder: the decoder is a **receiver** that determines how much information is accepted from each **sender**. For example, in the NMT model of the previous section, there is only one sender $\mathbf{h}_m$, and so the receiver receives all the information the sender sends. By contrast, in the NMT model equipped with the attention mechanism, there are m senders $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ and the receiver receives information according to a distribution of preferences over the senders.

It is straightforward to introduce the attention model into the process of word prediction. To make use of both source-side and target-side information at each decoding step, we slightly modify the definition of $\mathbf{s}_i$ to include the context vector corresponding to the previous state $\mathbf{s}_{i-1}$, as follows:

$$\mathbf{s}_i = \text{RNN}(\mathbf{s}_{i-1}, \mathbf{c}_{i-1}, \mathbf{y}_i^e). \quad (5.25)$$

Compared with the model of Eq. (5.11), the model of Eq. (5.25) takes $\mathbf{c}_{i-1}$ as an additional input. Therefore, this model considers both the representation of the target-side words $y_1 \dots y_{i-1}$ (as encoded in $\mathbf{s}_{i-1}$ and $\mathbf{y}_i^e$) and the representation of the entire source-side sequence $x_1 \dots x_m$ (as encoded in $\mathbf{c}_{i-1}$). Then, the distribution of target words at position i can be conditioned on $\mathbf{s}_i$ as usual:

$$\Pr(\cdot | y_1, \dots, y_i, x_1, \dots, x_m) = \Pr(\cdot | \mathbf{s}_i), \quad (5.26)$$

where $\Pr(\cdot | \mathbf{s}_i)$ is generally a Softmax layer. This process is illustrated in Figure 5.4.

We now have a model for computing $\Pr(y_{i+1} | y_1 \dots y_i, x_1 \dots x_m)$. A brief outline of the key steps of this model is given by

1. Encode the source-side sequence as $\mathbf{h}_1 \dots \mathbf{h}_m$ where $\mathbf{h}_j = \text{RNN}(\mathbf{h}_{j-1}, \mathbf{x}_j^e)$.
2. Repeat the following procedure from $i = 1$ to $n - 1$.

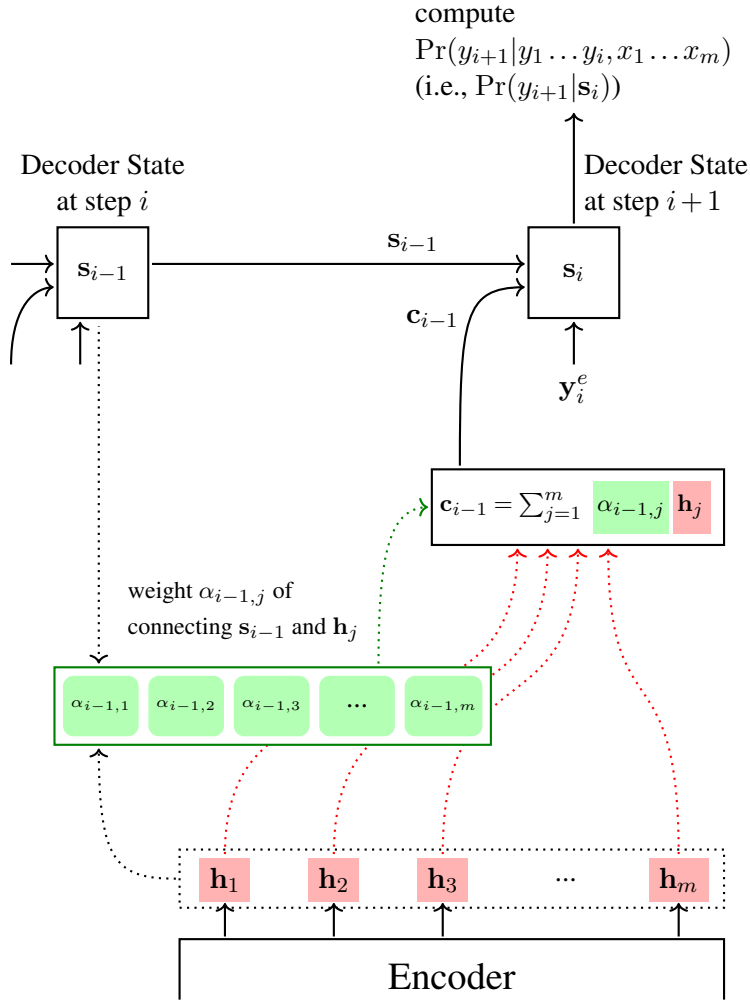


Figure 5.4: An attention model for NMT. Suppose we have obtained the representations $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ and the decoder state $\mathbf{s}_{i-1}$ up to this point. We wish to obtain the decoder state at the next step. To this end, we first compute attention weights by normalizing the attention scores between $\mathbf{s}_{i-1}$ and $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$, and then compute a context vector $\mathbf{c}_{i-1}$ by summing over $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ with the attention weights. A new decoder state $\mathbf{s}_i$ is created by taking the context vector $\mathbf{c}_{i-1}$, the previous state $\mathbf{s}_{i-1}$, and the word representation $\mathbf{y}_i^e$. $\mathbf{s}_i$ will be used as a condition for predicting a distribution of words at step $i+1$.

- Compute the alignment score $a(\mathbf{s}_{i-1}, \mathbf{h}_j)$ for each j .
- Compute the attention weights $\{\alpha_{i-1,1}, \dots, \alpha_{i-1,m}\}$ where $\alpha_{i-1,j} = \frac{\exp(a(\mathbf{s}_{i-1}, \mathbf{h}_j))}{\sum_{j'=1}^m \exp(a(\mathbf{s}_{i-1}, \mathbf{h}_{j'}))}$.
- Compute the context vector $\mathbf{c}_{i-1} = \sum_{j=1}^m \alpha_{i-1,j} \cdot \mathbf{h}_j$.
- Compute the target-side state $\mathbf{s}_i = \text{RNN}(\mathbf{s}_{i-1}, \mathbf{c}_{i-1}, \mathbf{y}_i^e)$.

- e. Compute the distribution of target-side words $\Pr(\cdot | \mathbf{s}_i)$.
- f. Compute $\Pr(y_{i+1} | y_1 \dots y_i, x_1 \dots x_m) = \Pr(y_{i+1} | \mathbf{s}_i)$ for a given word y_{i+1} (as in training), or select the most likely word $\hat{y}_{i+1} = \arg \max_{y_{i+1}} \Pr(y_{i+1} | y_1 \dots y_i, x_1 \dots x_m)$ (as in testing).

In real-world systems, this basic model can be modified to better predict the target-side words. For example, we can introduce fusion layers to combine $\mathbf{s}_i$, $\mathbf{c}_{i-1}$, and $\mathbf{y}_i^e$ before the Softmax layer so that we have a deeper model for prediction [Bahdanau et al., 2014]. Another commonly used approach is to stack multiple RNN layers on the target-side. In this case, one can perform attention in either each layer of the stack [Wu et al., 2016] or the top-most layer of the stack [Luong et al., 2015]. See Section 5.3.4 for more information about multi-layer approaches to attention.

5.3.2 The QKV Attention

Because the attention mechanism is such a powerful approach, many variants have been developed. Perhaps the most widely used approach is to reframe the attention problem as one of matching a query against a set of key-value pairs. It lays the foundation for the well-known sequence model — Transformer [Vaswani et al., 2017].

Here we assume that there are a number of key-value pairs $\{(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)\}$ and a query $\mathbf{q}$. The goal of the **query-key-value attention** (or **QKV attention**) model is to aggregate values by considering the correspondence between the query and the keys. This is a standard search problem in database systems in which information is returned in its original form or a new form when it matches the query. In the QKV attention, the result of the search is not a single value in $\{\mathbf{v}_1, \dots, \mathbf{v}_m\}$ but instead a weighted combination of these values. This is the key difference between this attention model and conventional search models.

Formally, the result of the QKV attention is defined to be

$$\mathbf{c} = \sum_{j=1}^m \alpha_j \mathbf{v}_j, \quad (5.27)$$

where

$$\alpha_j = \frac{\exp(\mathbf{q}\mathbf{k}_j^\top / \beta)}{\sum_{j'=1}^m \exp(\mathbf{q}\mathbf{k}_{j'}^\top / \beta)} \quad (5.28)$$

is the attention weight. It turns out that the above model has precisely the same general form as the model described in the previous subsection, and $\mathbf{c}$ can be simply viewed as a context vector.

While the basic form of the QKV attention is not something new, it can handle a variety of problems by giving $\mathbf{q}$, $\mathbf{k}_j$ and $\mathbf{v}_j$ appropriate meanings. Here we consider a more general case where there are n queries $\{\mathbf{q}_1, \dots, \mathbf{q}_n\}$ and n output vectors $\{\mathbf{c}_1, \dots, \mathbf{c}_n\}$. To simplify

notation, we use $\mathbf{Q}$ to denote a matrix whose i -th row is $\mathbf{q}_i$:

$$\mathbf{Q} = \begin{bmatrix} \mathbf{q}_1 \\ \vdots \\ \mathbf{q}_n \end{bmatrix}. \quad (5.29)$$

Likewise, we can define $\mathbf{K} = \begin{bmatrix} \mathbf{k}_1 \\ \vdots \\ \mathbf{k}_m \end{bmatrix}$, $\mathbf{V} = \begin{bmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_m \end{bmatrix}$, and $\mathbf{C} = \begin{bmatrix} \mathbf{c}_1 \\ \vdots \\ \mathbf{c}_n \end{bmatrix}$. Then, the attention model can be formulated as

$$\mathbf{C} = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\beta}\right)\mathbf{V}. \quad (5.30)$$

Figure 5.5 shows an illustration of this equation. Note that $\text{Softmax}(\frac{\mathbf{Q}\mathbf{K}^\top}{\beta})$ is the row-wise Softmax function, which produces a matrix of attention weights

$$\text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\beta}\right) = \begin{bmatrix} \alpha_{1,1} & \dots & \alpha_{1,m} \\ \vdots & & \vdots \\ \alpha_{n,1} & \dots & \alpha_{n,m} \end{bmatrix}, \quad (5.31)$$

where a row vector $[\alpha_{i,1} \dots \alpha_{i,m}]$ represents a distribution over the indices of the values. We can then expand Eq. (5.30) to facilitate understanding of the model

$$\begin{aligned} \mathbf{C} &= \begin{bmatrix} \mathbf{c}_1 \\ \vdots \\ \mathbf{c}_n \end{bmatrix} \\ &= \begin{bmatrix} \sum_{j=1}^m \alpha_{1,j} \mathbf{v}_j \\ \vdots \\ \sum_{j=1}^m \alpha_{n,j} \mathbf{v}_j \end{bmatrix} \\ &= \begin{bmatrix} \alpha_{1,1} & \dots & \alpha_{1,m} \\ \vdots & & \vdots \\ \alpha_{n,1} & \dots & \alpha_{n,m} \end{bmatrix} \begin{bmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_m \end{bmatrix}. \end{aligned} \quad (5.32)$$

In sequence-to-sequence modeling, $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ can be defined in several different ways. To describe the correspondence between the source-side and target-side sequences, one approach, called **encoder-decoder attention**, is to simply assume that

$$\mathbf{Q} = \begin{bmatrix} \mathbf{s}_1 \\ \vdots \\ \mathbf{s}_n \end{bmatrix}, \quad (5.33)$$

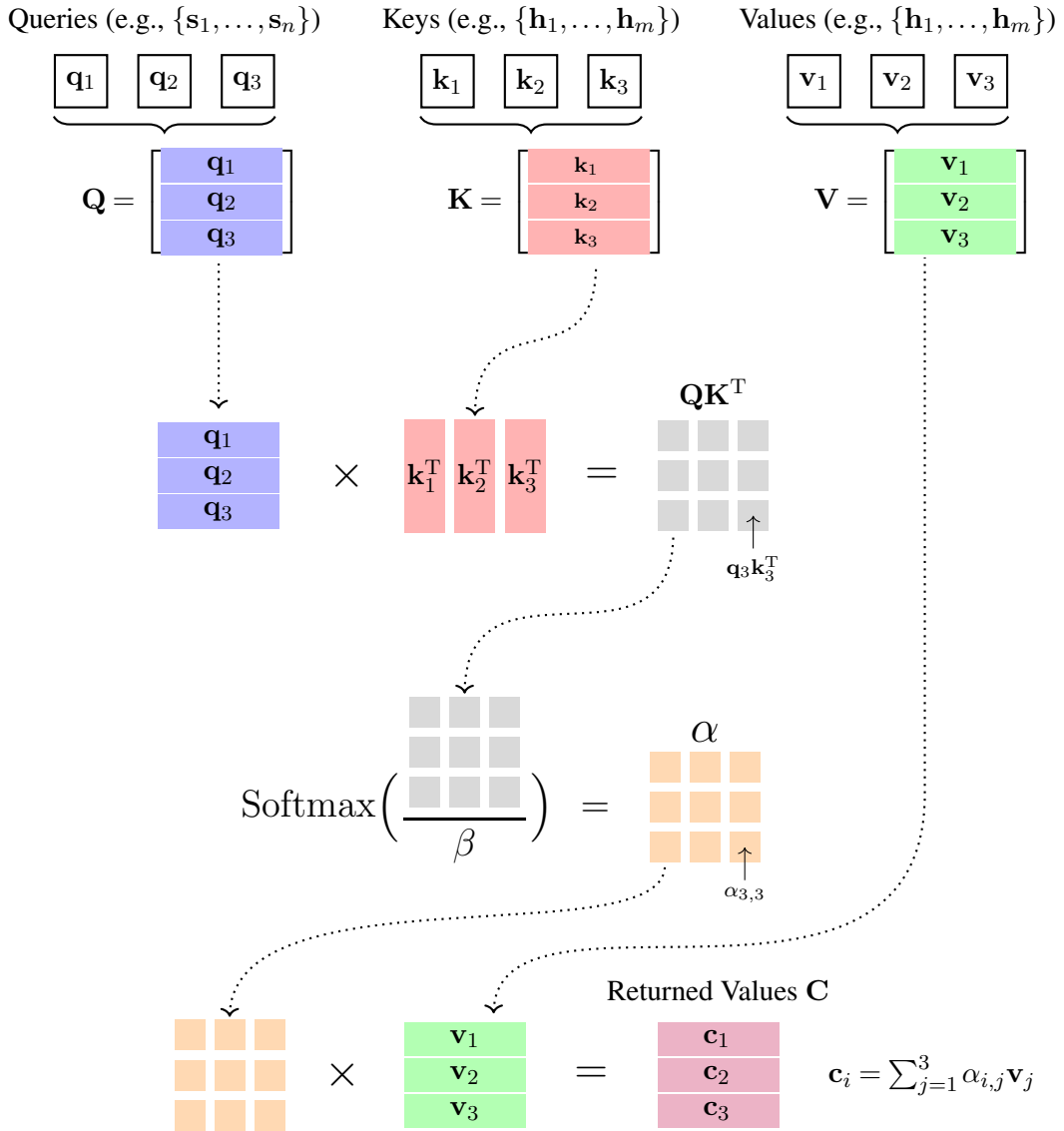


Figure 5.5: The QKV attention model for batches of queries ($\mathbf{Q}$), keys ($\mathbf{K}$), and values ($\mathbf{V}$). The figure shows a direct implementation of the formula $\mathbf{C} = \text{Softmax}\left(\frac{\mathbf{QK}^T}{\beta}\right)\mathbf{V}$. $\text{Softmax}\left(\frac{\mathbf{QK}^T}{\beta}\right)$ computes the attention weights by normalizing a scaled dot-product of $\mathbf{Q}$ and $\mathbf{K}^T$. This results in a matrix α in which a row vector represents the weights over different values. By multiplying α with $\mathbf{V}$, we obtain a sequence of new values, each expressing a weighted sum of the original values.

and

$$\mathbf{K} = \mathbf{V} = \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_m \end{bmatrix}. \quad (5.34)$$

In this case, $\mathbf{C}$ is a sequence of new representations of the source-side sequence given the representations of the target-side sequence. As with the model described in the previous subsection, each $\mathbf{c}_i \in \mathbf{C}$ can be used to predict the word y_{i+1} .

In addition to applying the model to sequence-to-sequence problems, another type of approach is to regard it as a sequence model, that is, we use the QKV attention to represent a sequence in one language. In this case, the QKV attention is also called **self-attention** which forms the basis of the well-known Transformer model [Vaswani et al., 2017]. Consider, for example, the sequence of states $\mathbf{h}_1 \dots \mathbf{h}_m$. The self-attention model assumes that

$$\mathbf{Q} = \mathbf{K} = \mathbf{V} = \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_m \end{bmatrix}. \quad (5.35)$$

Then, the output of the model is a sequence of representations $\mathbf{c}_1 \dots \mathbf{c}_m$. $\mathbf{c}_j$ is a representation which captures the correlations between $\mathbf{h}_j$ and all elements of the input sequence. We will see a more detailed discussion on this model in Chapter 6.

5.3.3 Multi-head Attention

Multi-head attention is a natural extension to the above models. The key idea is to perform attention in different subspaces of the representation space simultaneously rather than in a single representation space. To illustrate, consider a standard attention model that takes sequences of source-side and target-side states and outputs a sequence of new states, written as

$$\mathbf{c}_1 \dots \mathbf{c}_n = \text{Att}(\mathbf{h}_1 \dots \mathbf{h}_m, \mathbf{s}_1 \dots \mathbf{s}_n), \quad (5.36)$$

where $\mathbf{h}_j, \mathbf{s}_i, \mathbf{c}_i \in \mathbb{R}^{d_h}$, and $\text{Att}(\cdot)$ is the attention function. We can map $\mathbf{h}_j$ into τ vectors $\{\mathbf{h}_j^{[1]}, \dots, \mathbf{h}_j^{[\tau]}\}$ using the following linear transformations

$$\mathbf{h}_j^{[1]} = \mathbf{h}_j \mathbf{W}_h^{[1]}, \quad (5.37)$$

$$\vdots$$

$$\mathbf{h}_j^{[\tau]} = \mathbf{h}_j \mathbf{W}_h^{[\tau]}, \quad (5.38)$$

where $\mathbf{h}_j^{[1]}, \dots, \mathbf{h}_j^{[\tau]} \in \mathbb{R}^{\frac{d_h}{\tau}}$, and $\mathbf{W}_h^{[1]}, \dots, \mathbf{W}_h^{[\tau]} \in \mathbb{R}^{d_h \times \frac{d_h}{\tau}}$.

Similarly, we can map $\mathbf{s}_i$ into τ vectors $\{\mathbf{s}_i^{[1]}, \dots, \mathbf{s}_i^{[\tau]}\}$. We then define τ **feature subspaces** in which the attention function is performed independently. For the k -th feature subspace, we have

$$\mathbf{c}_1^{[k]} \dots \mathbf{c}_n^{[k]} = \text{Att}(\mathbf{h}_1^{[k]} \dots \mathbf{h}_m^{[k]}, \mathbf{s}_1^{[k]} \dots \mathbf{s}_n^{[k]}). \quad (5.39)$$

The output of the model is a sequence of d_h -dimensional vectors, each of which is obtained by concatenating the vectors that are produced in all these feature subspaces, followed by a

linear transformation. This procedure is given by

$$\mathbf{c}_1 = [\mathbf{c}_1^{[1]}, \dots, \mathbf{c}_1^{[\tau]}] \mathbf{W}_c, \quad (5.40)$$

...

$$\mathbf{c}_n = [\mathbf{c}_n^{[1]}, \dots, \mathbf{c}_n^{[\tau]}] \mathbf{W}_c, \quad (5.41)$$

where $\mathbf{W}_c \in \mathbb{R}^{d_h \times d_h}$.

Following the notation used in the previous subsection, we can express a sequence of vectors as a matrix, e.g., $\mathbf{H} = \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_m \end{bmatrix} \in \mathbb{R}^{m \times d_h}$, $\mathbf{S} = \begin{bmatrix} \mathbf{s}_1 \\ \vdots \\ \mathbf{s}_n \end{bmatrix} \in \mathbb{R}^{n \times d_h}$, and $\mathbf{C} = \begin{bmatrix} \mathbf{c}_1 \\ \vdots \\ \mathbf{c}_n \end{bmatrix} \in \mathbb{R}^{n \times d_h}$.

Using this notation, we rewrite Eq. (5.36) as

$$\mathbf{C} = \text{Att}(\mathbf{H}, \mathbf{S}). \quad (5.42)$$

To give a formal definition of multi-head attention, we first introduce the split and merge functions. The split function divides each row vector of a matrix into a number of sub-vectors, resulting in a 3D tensor. For example, splitting an $m \times d_h$ matrix $\mathbf{A}$ with τ produces a $\tau \times m \times \frac{d_h}{\tau}$ tensor³

$$\mathbf{A}_{\text{heads}} = \text{Split}(\mathbf{A}, \tau). \quad (5.43)$$

The merge function performs the inverse operation of the split function. Given a $\tau \times n \times \frac{d_h}{\tau}$ tensor (say $\mathbf{A}_{\text{heads}}$), it merges each group of τ $\frac{d_h}{\tau}$ -dimensional sub-arrays in the form

$$\mathbf{A}_{\text{merge}} = \text{Merge}(\mathbf{A}_{\text{heads}}, \tau). \quad (5.44)$$

Thus the form of multi-head attention is given by

$$\begin{aligned} \mathbf{C} &= \mathbf{C}_{\text{merge}} \mathbf{W}_c \\ &= \text{Merge}(\mathbf{C}_{\text{heads}}, \tau) \mathbf{W}_c \\ &= \text{Merge}(\text{Att}(\mathbf{H}_{\text{heads}}, \mathbf{S}_{\text{heads}}), \tau) \mathbf{W}_c \end{aligned} \quad (5.45)$$

$$\mathbf{H}_{\text{heads}} = \text{Split}(\mathbf{H} \mathbf{W}_h, \tau) \quad (5.46)$$

$$\mathbf{S}_{\text{heads}} = \text{Split}(\mathbf{S} \mathbf{W}_s, \tau), \quad (5.47)$$

where $\mathbf{W}_h, \mathbf{W}_s \in \mathbb{R}^{d_h \times d_h}$ are the parameters. $\text{Split}(\mathbf{H} \mathbf{W}_h, \tau)$ implements the projections of Eqs. (5.37-5.38) for all $\mathbf{h}_j$. Likewise, $\text{Split}(\mathbf{S} \mathbf{W}_s, \tau)$ has an analogous interpretation. Note that here $\text{Att}(\cdot)$ is extended to deal with multi-head inputs. See Figure 5.6 for an illustration of this model.

Multi-head attention is a very general approach that can be extended to many models. As a simple example of this extension, consider the QKV attention model discussed in the previous

³An $a \times b \times c$ tensor can be treated as an array of a matrices whose shapes are $b \times c$.

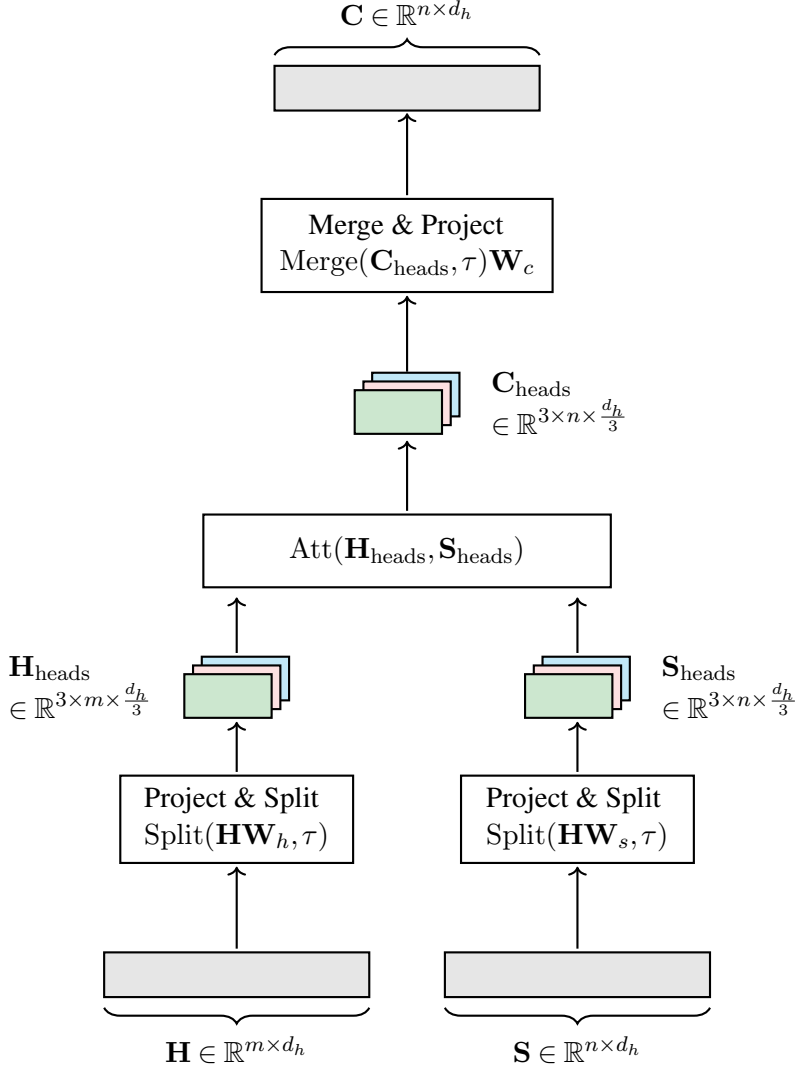


Figure 5.6: An attention model with $\tau = 3$ heads. First, we transform the input matrices into multi-head representations, i.e., 3D tensors $\mathbf{H}_{\text{heads}} \in \mathbb{R}^{3 \times m \times \frac{d_h}{3}}$ and $\mathbf{S}_{\text{heads}} \in \mathbb{R}^{3 \times n \times \frac{d_h}{3}}$. These tensors are then fed into an attention model. The output of this model is a tensor $\mathbf{C}_{\text{heads}} \in \mathbb{R}^{3 \times n \times \frac{d_h}{3}}$. We then merge the heads of $\mathbf{C}_{\text{heads}}$, followed by a linear transformation. Finally, we obtain n vectors of size d_h , represented by an $n \times d_h$ matrix.

subsection. Let $\text{Att}_{\text{QKV}}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ be the attention function, and $\mathbf{Q} \in \mathbb{R}^{d_k}, \mathbf{K} \in \mathbb{R}^{d_k}, \mathbf{V} \in \mathbb{R}^{d_v}$

be the inputs. The multi-head QKV attention model is given by

$$\mathbf{C} = \text{Merge}(\text{Att}_{\text{QKV}}(\mathbf{Q}_{\text{heads}}, \mathbf{K}_{\text{heads}}, \mathbf{V}_{\text{heads}}))\mathbf{W}_c, \quad (5.48)$$

$$\mathbf{Q}_{\text{heads}} = \text{Split}(\mathbf{Q}\mathbf{W}_q, \tau), \quad (5.49)$$

$$\mathbf{K}_{\text{heads}} = \text{Split}(\mathbf{K}\mathbf{W}_k, \tau), \quad (5.50)$$

$$\mathbf{V}_{\text{heads}} = \text{Split}(\mathbf{V}\mathbf{W}_v, \tau), \quad (5.51)$$

where $\mathbf{W}_q \in \mathbb{R}^{d_k \times d_k}$, $\mathbf{W}_k \in \mathbb{R}^{d_k \times d_k}$, $\mathbf{W}_v \in \mathbb{R}^{d_v \times d_v}$, $\mathbf{W}_c \in \mathbb{R}^{d_v \times d_v}$ are the model parameters.

One advantage of multi-head attention is that each feature subspace describes a different perspective of attention (call it an **attention head** or **head** for short). Therefore, the concatenation of the outputs over these heads represents an ensemble of attention models that deal with different parts of the data. This is similar to learning a group of models independently and combining them to form a stronger model. Such approaches have been proven to be useful in many problems [Opitz and Maclin, 1999; Zhou, 2012]. Note that the multi-head attention models discussed here are parameterized by the linear projections on the input and output spaces. The use of these linear projections is generally helpful as the models become deeper and can describe more complex problems.

From an architecture design perspective, multi-head attention falls into a broad class of neural networks — those involving a number of branches of layer stacks for dealing with the same input (call them **multi-branch neural networks**). However, unlike conventional approaches, which require different model architectures for different branches, the multi-head attention approach is based on a single model for all the heads. As a result, such systems are very efficient in practice because the attention procedure can run in parallel over these heads.

5.3.4 Multi-layer Attention

So far we have considered the case of **single-layer attention** — the output of the attention models can be written as a linear combination of the source-side representations. Now we extend it in a natural way to **multi-layer attention** in which the single-layer attention procedure runs a number of times for forming a “deeper” attention model.

To do this, a multi-layer neural network is created on the target-side. The model architecture is regular. We stack a number of attention layers, each interacting with the source-side sequence and feeding its output to the next layer. In an attention layer, we perform attention as usual. For the l -th layer in the stack, this step takes the source-side sequence (denoted by $\mathbf{h}_1 \dots \mathbf{h}_m$) as well as the output of the previous layer (denoted by $\mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}$), and produces a sequence of vectors by

$$\mathbf{c}_1^l \dots \mathbf{c}_n^l = \text{Att}(\mathbf{h}_1 \dots \mathbf{h}_m, \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}), \quad (5.52)$$

where $\text{Att}(\cdot)$ could be any attention function described in this chapter.

Then, we create another neural network $f(\cdot)$ to give more modeling power to the model.

The output of the attention layer is thus defined to be

$$\mathbf{s}_1^l \dots \mathbf{s}_n^l = f(\mathbf{c}_1^l \dots \mathbf{c}_n^l, \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}). \quad (5.53)$$

$f(\cdot)$ can be designed in many ways [Sukhbaatar et al., 2015; Wu et al., 2016; Vaswani et al., 2017]. A popular choice is to define $f(\cdot)$ as a feed-forward neural network with a residual connection, given by

$$f(\mathbf{c}_1^l \dots \mathbf{c}_n^l, \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}) = \text{FFN}(\mathbf{c}_1^l \dots \mathbf{c}_n^l) + \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}. \quad (5.54)$$

Substituting for the vectors $\mathbf{c}_1^l \dots \mathbf{c}_n^l$, using Eq. (5.52), the output of layer l is written in the form

$$\mathbf{s}_1^l \dots \mathbf{s}_n^l = \text{FFN}(\text{Att}(\mathbf{h}_1 \dots \mathbf{h}_m, \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1})) + \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}. \quad (5.55)$$

As with the models in the previous subsections, it is convenient to use a more compact notation by expressing a sequence of vectors as a matrix. Thus this model can be given in another form

$$\mathbf{S}^l = \text{FFN}(\text{Att}(\mathbf{H}, \mathbf{S}^{l-1})) + \mathbf{S}^{l-1}. \quad (5.56)$$

Here $\text{FFN}(\cdot)$ is generally a multi-layer neural network with non-linear activation functions. The identity mapping (i.e., $+\mathbf{S}^{l-1}$) creates a direct path from the input to the output of the layer, making it easier to train a deep neural network.

Figure 5.7 shows the architecture of this model. The attention model starts with the initial representation of the target-side sequence, that is, $\mathbf{S}^0 = \mathbf{S} = \begin{bmatrix} \mathbf{s}_1 \\ \vdots \\ \mathbf{s}_n \end{bmatrix}$. If there are L attention layers, then the final output will be $\mathbf{S}^L$.

5.3.5 Remarks

Above we considered a basic attention model and a series of refinements. The literature on attention and related topics contains a wide range of methods for modeling how a system concentrates on different parts of the input, as well as many applications of such models. This subsection discusses some interesting issues.

1. Word Alignment vs. Attention

Attention is related to a long line of research on (word) alignment which models the correspondence between two groups of language units. In NLP, alignment is a very general concept that is used to refer to several problems. For example, most statistical machine translation systems are trained on bilingual texts with word-to-word alignments [Koehn et al., 2003; Chiang, 2005]. **Word alignment** models are thus developed to generate links between words in two sentences [Vogel et al., 1996; Och and Ney, 2003; Taskar et al., 2005; Dyer et al., 2013].

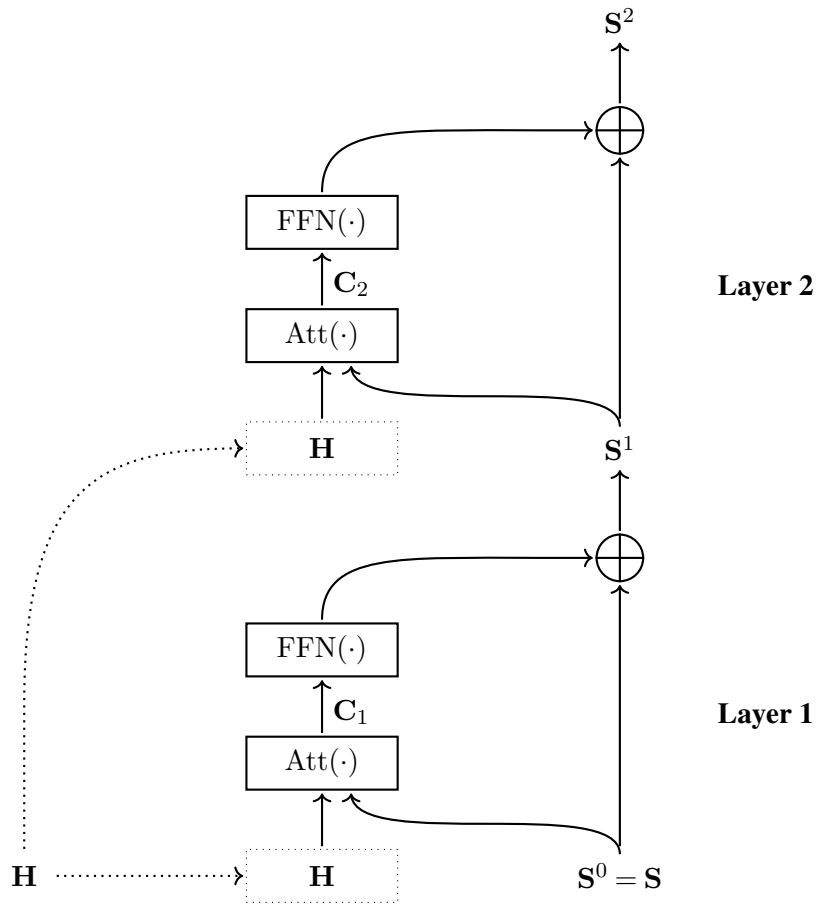


Figure 5.7: A 2-layer attention model. These layers take the same “key-value” pairs (i.e., $\mathbf{H}$) but each takes a different “query” (i.e., $\mathbf{S}^l$). The attention model is standard: context vectors $\mathbf{C}^l$ are generated by taking both $\mathbf{H}$ and $\mathbf{S}^l$. A feed-forward neural network is built to transform $\mathbf{C}^l$, followed by an addition of $\mathbf{S}^l$. So this model can be formulated as $\mathbf{S}^l = \text{FFN}(\text{Att}(\mathbf{H}, \mathbf{S}^{l-1})) + \mathbf{S}^{l-1}$. $\mathbf{S}^l$ is then used in the next layer as the query, that is, layer $l + 1$ takes $\mathbf{H}$ and $\mathbf{S}^l$, and repeats the above process. The output of the last layer can be viewed as a deeper representation of $\mathbf{H}$ for $\mathbf{S}$.

While the outputs of these systems are discrete variables, the underlying models are typically probabilistic and may involve continuous representations. Therefore, the correspondence between word alignment and the attention models discussed here is apparent because they both assign weights to pairs of words.

Note that despite the similarity between word alignment and attention problems, their goals are different. In most cases word alignment models are used as individual systems to produce alignment results for downstream systems, whereas attention models are typically treated as components of bigger systems and do not work alone (see Figure 5.8 for a comparison of these models). As a result, they are suited to different types of sequence-to-sequence systems in practice: word alignment is one step in cascaded systems, and attention represents intermediate

states of a neural network.

Nevertheless, word alignment and attention are two related problems, and can help each other in some cases. For example, one way to see how an attention model behaves is to induce word alignments from it and measure the quality of these word alignments [Tu et al., 2016; Li et al., 2019; Garg et al., 2019]. Also, systems equipped with the attention mechanism can be guided by external word alignment resources [Mi et al., 2016b; Liu et al., 2016b].

2. Introducing Priors

As discussed in Section 5.3.1, the attention models implicitly define an attention distribution over $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ which is used to compute a weighted sum of these representations. This distribution is expressed in terms of the alignment weights and is learned as part of the model. In addition to learning the attention distribution in an end-to-end fashion, we can also define it based on prior knowledge of how attention is allocated in sequences.

One approach is to directly impose some structure on the distribution. A straightforward example is to restrict attention to a local span of the sequence and discard the rest. Let $[\rho_i - D, \rho_i + D]$ be a $(2D + 1)$ -word window centered at position ρ_i of the source-side sequence. We can restrict attention from $\mathbf{s}_i$ to $\mathbf{h}_{\rho_i - D} \dots \mathbf{h}_{\rho_i + D}$, yielding a local context vector formulated as:

$$\mathbf{c}_i = \text{Att}(\mathbf{h}_{\rho_i - D} \dots \mathbf{h}_{\rho_i + D}, \mathbf{s}_i). \quad (5.57)$$

This approach is also called **local attention**. Determining ρ_i is similar to the **reordering problem** in machine translation. For translation between languages with similar word orders, it is common to assume that the translation is monotonic and ρ_i is linear with respect to i [Koehn, 2004], e.g., $\rho_i = \lfloor m \frac{i}{n} \rfloor$ or $\lceil m \frac{i}{n} \rceil$. An alternative method is to use a neural network to predict a relative position in the source-side sequence (denoted by r_i) [Luong et al., 2015]. ρ_i can then be defined to be $\lfloor mr_i \rfloor$ or $\lceil mr_i \rceil$.

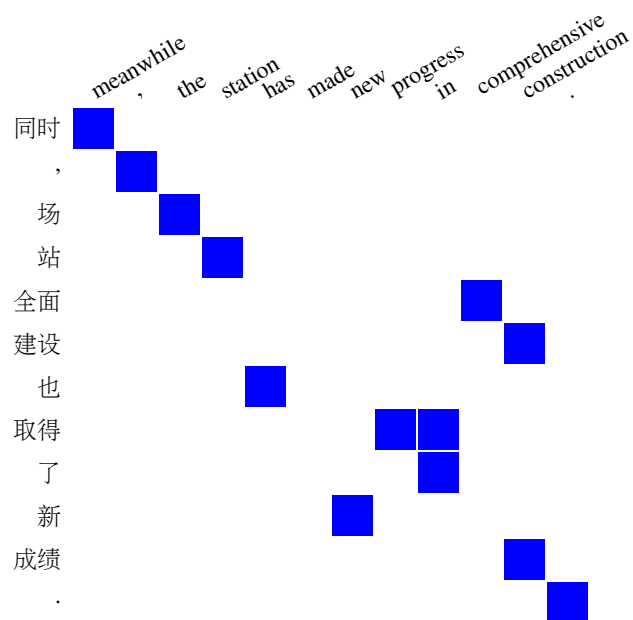
In another thread of research, a new distribution is constructed by combining the original attention distribution and some prior distribution. The simplest such distribution takes the form of linear interpolation

$$\widetilde{\text{Pr}}(\mathbf{h}_j | \mathbf{s}_i) = \eta \cdot \text{Pr}(\mathbf{h}_j | \mathbf{s}_i) + (1 - \eta) \cdot \text{Prior}, \quad (5.58)$$

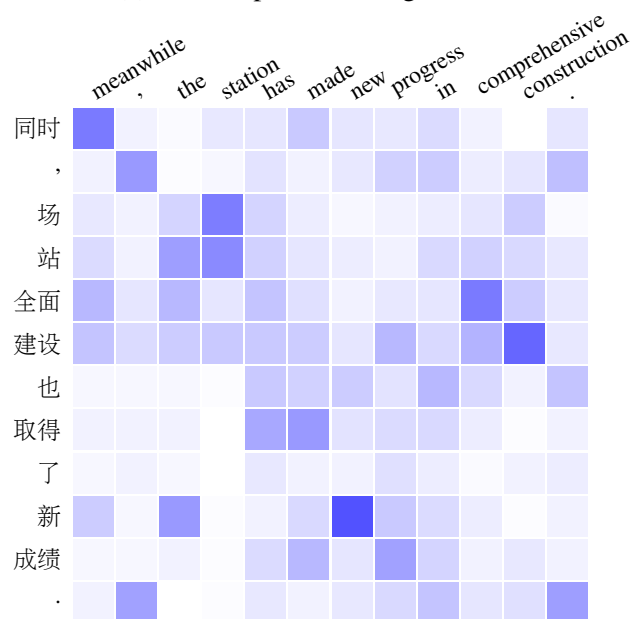
where Prior denotes the prior distribution, and η denotes the interpolation coefficient. When $\eta = 1$, it is a standard attention model. By contrast, when $\eta = 0$, the attention is completely dependent on the prior [You et al., 2020].

The prior can be chosen in many ways. A simple choice is to assume Prior to be a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$. This provides a clear intuitive interpretation: attention is concentrated at a focal point in the sequence and decays as the distance from this point increases. To determine the mean and variance of the Gaussian distribution, we can use the same technique described above, for example, we develop two neural networks to compute the mean and variance, respectively.

The interpolation can also be considered an intermediate step in computing the attention



(a) Heat map of word alignments



(b) Heat map of attention weights

Figure 5.8: Heat maps of a word alignment model and an attention model for a pair of Chinese and English sentences. The heat maps are represented as shaded grids in which each cell describes the correspondence of a pair of Chinese and English words. This correspondence is shown on a color scale ranging from white denoting a weight of 0 to pure blue denoting a weight of 1.

distribution. For example, consider the QKV attention discussed in Section 5.3.2. The interpolation can be applied to the query-key dot-product [Yang et al., 2018a; Guo et al., 2019]. To this end, we can modify Eq. (5.28) in the following form

$$\begin{aligned}\alpha_j &= \text{Softmax} \left(\frac{\mathbf{q}\mathbf{k}_j^\top}{\beta} + \eta \text{Prior} \right) \\ &= \text{Softmax} \left(\frac{\mathbf{s}_i \mathbf{h}_j^\top}{\beta} + \eta \text{Prior} \right).\end{aligned}\tag{5.59}$$

As $\frac{\mathbf{q}\mathbf{k}_j^\top}{\beta}$ (or $\frac{\mathbf{s}_i \mathbf{h}_j^\top}{\beta}$) is not constrained to $[0, 1]$, Prior is re-scaled by a hyper-parameter η .

Sometimes, priors arise when knowledge about attention comes from external sources. As discussed above, incorporating word alignments into attention models is one of the simplest ways to do this. The idea can be extended to make use of more structural information, such as fertility and coverage [Cohn et al., 2016; Feng et al., 2016; Tu et al., 2016], or more task-specific constraints, such as monotonic alignments between input and output sequences [Raffel et al., 2017; Chiu and Raffel, 2018]. Also, as with syntactic machine translation systems, parse trees can be used to bias the process of attention as an auxiliary input. For example, dependency trees are a widely used source of information in modeling word correspondence for either sequence-to-sequence [Chen et al., 2018] or sequence modeling problems [Zhang et al., 2020b; Nguyen et al., 2020; Xu et al., 2021].

Because standard attention mechanisms can be computationally expensive in large-scale applications, researchers have developed efficient alternatives by introducing stronger inductive biases into the model design [Tay et al., 2020]. This line of research falls under the broader category of efficient NLP methods. Chapter 6 discusses these methods in detail.

3. Attention in Memory Networks

As well as being of great interest in sequence-to-sequence systems, the attention mechanism is extensively used in memory-based neural models [Sukhbaatar et al., 2015; Graves et al., 2014; Kumar et al., 2016]. As discussed in Chapter 4, a memory system maintains a collection of data items, allowing the model to retain and retrieve information. Given a query, the system computes a match score between the query and the key of each data item. This procedure is also known as **addressing** [Graves et al., 2014]. Such addressing is typically implemented by first representing the query and the data items as real-valued vectors, and then calculating a weight based on a similarity measure between them. The result of the retrieval is a weighted sum of the data items. This formalism aligns perfectly with the QKV attention model discussed in Section 5.3.2.

Given the conceptual link between attention and memory mechanisms, at least from a computational, if not psychological perspective, we can view attention as a process of retrieving information from a memory bank (i.e., $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$) using a given query (i.e., $\mathbf{s}_i$). Thus we can interpret a sequence-to-sequence system with the attention mechanism as follows. On the source-side, we store information in a memory represented as a sequence of vectors $\mathbf{h}_1 \dots \mathbf{h}_m$.

Then, we retrieve from this memory to recover step by step the source-side information on the target-side.

4. Beyond Sequence-to-Sequence Problems

While we restrict our discussion in this section to the problem of transforming one sequence into another, the general attention mechanism can be applied to address a broader range of tasks. As mentioned in Section 5.3.2, and as will be discussed in Chapter 6, a well-known variant of this approach is self-attention. In self-attention, the QKV attention model is used as a sequence model. It provides a general framework for modeling the interactions and dependencies between input variables, which makes it applicable to a variety of problems. For example, we can concatenate $\mathbf{h}_1 \dots \mathbf{h}_m$ and $\mathbf{s}_1 \dots \mathbf{s}_n$ into a single combined sequence $\mathbf{h}_1 \dots \mathbf{h}_m \mathbf{s}_1 \dots \mathbf{s}_n$, and apply the model to this new sequence. In this way, self-attention can be naturally extended to a sequence-to-sequence model [Lample and Conneau, 2019; Raffel et al., 2020]. Such an approach is effective even when $\mathbf{h}_1 \dots \mathbf{h}_m$ and $\mathbf{s}_1 \dots \mathbf{s}_n$ represent different modalities. For instance, if we use $\mathbf{h}_1 \dots \mathbf{h}_m$ to represent text and $\mathbf{s}_1 \dots \mathbf{s}_n$ to represent an image, we obtain a multi-modal model that fuses textual and visual representations by performing self-attention across them [Chen et al., 2020].

Another approach to learning joint representations of sequences is to build multiple attention models, allowing each sequence to attend to the other. An example of this approach is **co-attention**, which has been widely adopted in multi-modal language processing [Lu et al., 2016]. In tasks such as **visual question answering (VQA)**, we aim to determine which parts of an image correspond to a specific word in the question, and conversely, which words in the question align with a given visual region. To achieve this, we construct two attention models: one for image-to-text attention, and another for text-to-image attention. The outputs of both models serve as joint representations of the image and text, and thus can be used as features for answer prediction.

The attention models discussed in this section are invariant to the order of their inputs. While this permutation invariance poses a challenge for modeling sequential data, it can be addressed by explicitly encoding position information into the inputs themselves (see Chapters 4 and 6). On the other hand, the simplicity of this formulation makes these models broadly applicable: the input data need not be sequential. As a result, attention mechanisms can be directly applied to more complex data structures, such as graphs [Veličković et al., 2018; Lee et al., 2019].

5.4 Search

Search is a fundamental problem in artificial intelligence and plays a crucial role in many NLP systems. The search problem presents a computational challenge here because the number of hypotheses in the search space increases exponentially with the length of the sequence and the size of the target vocabulary. Exhaustive search in this context is computationally prohibitive. Therefore, real-world systems typically employ search algorithms or heuristics to ensure that optimal or near-optimal solutions can be found in acceptable computational time.

For many practical sequence-to-sequence applications, the search problem, often referred to as inference or decoding, can be formulated as follows:

$$\hat{\mathbf{y}} = \arg \max_{\mathbf{y} \in \Omega} \text{Score}(\mathbf{x}, \mathbf{y}), \quad (5.60)$$

where $\text{Score}(\mathbf{x}, \mathbf{y})$ is a scoring function that measures the quality of $\mathbf{y}$ given $\mathbf{x}$.

This formulation differs slightly from Eq. (5.2). First, we use $\text{Score}(\mathbf{x}, \mathbf{y})$ instead of $\Pr(\mathbf{y}|\mathbf{x})$ as the objective function. While a standard approach to training sequence-to-sequence models is to maximize $\Pr(\mathbf{y}|\mathbf{x})$ (or $\Pr(\mathbf{x}, \mathbf{y})$), we often need to account for task-specific issues, such as length bias, at inference time. It is therefore common to incorporate other penalty and reward terms alongside $\Pr(\mathbf{y}|\mathbf{x})$ to define the final search objective (see Section 5.4.1). A second difference arises from the search space itself, which is constrained to a subset Ω . In practice, Ω is a pruned search space containing a relatively small number of highly probable hypotheses. This is typically achieved through pruning techniques and advanced search algorithms (see Section 5.4.2). In this section, we explore solutions to these challenges and discuss several refinements. While these methods are largely motivated by developments in machine translation, the concepts are general and apply broadly to most text generation tasks.

5.4.1 The Length Problem

Recall from Section 5.2.2 that the probability of the target-side sequence $\mathbf{y}$ given the source-side sequence $\mathbf{x}$ can be written as the product of the probabilities of each word y_i given both the previously generated words $y_0 \dots y_{i-1}$ and $\mathbf{x}$. Here, we re-express Eq. (5.14) using a simpler notation:

$$\Pr(\mathbf{y}|\mathbf{x}) = \prod_{i=1}^n \Pr(y_i | \mathbf{y}_{<i}, \mathbf{x}), \quad (5.61)$$

where $\mathbf{y}_{<i}$ denotes the sequence $y_0 \dots y_{i-1}$. This can be equivalently expressed in terms of log probabilities

$$\log \Pr(\mathbf{y}|\mathbf{x}) = \sum_{i=1}^n \log \Pr(y_i | \mathbf{y}_{<i}, \mathbf{x}). \quad (5.62)$$

While this simple form of modeling offers clear advantages for practical NLP applications, it unfortunately introduces a strong bias toward shorter target sequences. Therefore, directly using $\text{Score}(\mathbf{x}, \mathbf{y}) = \Pr(\mathbf{y}|\mathbf{x})$ or $\log \Pr(\mathbf{y}|\mathbf{x})$ as the search objective is problematic, as the output is highly likely to be an excessively short sequence (e.g., a sequence comprising only one or two words). This problem is a direct consequence of the model’s inductive bias. From a supervised learning perspective, this issue arises because **teacher forcing** is used during training: the model is exposed to ground-truth target sequences and is forced to predict them. By contrast, during inference on test data, we must explore a large hypothesis space of $\mathbf{y}$ with varying lengths, comparing them using a scoring function that effectively differs from the one

optimized during training.

This problem can be addressed through a technique called **length reward**, which gives bonuses to longer sequences by adding a term to $\text{Score}(\mathbf{x}, \mathbf{y})$ [He et al., 2016]. As discussed in Chapter 3, a commonly used form of length reward is given by

$$\text{Score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda \cdot n. \quad (5.63)$$

Here the length reward term is the length of $\mathbf{y}$ (i.e., $n = |\mathbf{y}|$), weighted by the parameter $\lambda > 0$. Improvements to this approach involve replacing n with an estimated sequence length by using a length prediction model. For example, we can bound the reward in the following form [Huang et al., 2017; Yang et al., 2018b]

$$\text{Score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda \cdot \max(l_p, n), \quad (5.64)$$

where l_p is a predicted length, generally defined to be a scaled length of $\mathbf{x}$, that is, $l_p = \text{scalar}_p \cdot m$.

If we substitute the log probability $\log \Pr(\mathbf{y}|\mathbf{x})$ given by Eq. (5.62) into Eq. (5.63), we obtain

$$\begin{aligned} \text{Score}(\mathbf{x}, \mathbf{y}) &= \sum_{i=1}^n \log \Pr(y_i | \mathbf{y}_{<i}, \mathbf{x}) + \lambda \cdot n \\ &= \sum_{i=1}^n [\log \Pr(y_i | \mathbf{y}_{<i}, \mathbf{x}) + \lambda]. \end{aligned} \quad (5.65)$$

Thus, we can interpret the length reward term as a reward to each word y_i . Such a method has been widely used in **statistical machine translation (SMT)** systems in which the rewards are treated as features of a log-linear model [Koehn et al., 2003; Chiang, 2007]. To find an appropriate value of λ , we can either use minimum error rate training [Och, 2003], following the convention in SMT, or use gradient-based methods as in neural network-based systems [Murray and Chiang, 2018].

A second approach to biasing search towards longer sequences, called **length normalization**, is to divide $\log \Pr(\mathbf{y}|\mathbf{x})$ by a length correction term, written in the following form

$$\text{Score}(\mathbf{x}, \mathbf{y}) = \frac{\log \Pr(\mathbf{y}|\mathbf{x})}{n_{\text{correct}}}. \quad (5.66)$$

A simple example of this model is to define the length correction term as the sequence length [Jean et al., 2015], like this

$$\begin{aligned} n_{\text{correct}} &= n \\ &= |\mathbf{y}|. \end{aligned} \quad (5.67)$$

In this case, $\frac{\log \Pr(\mathbf{y}|\mathbf{x})}{n} = \frac{\sum_{i=1}^n \log \Pr(y_i | \mathbf{y}_{<i}, \mathbf{x})}{n}$ can be viewed as the log-scale geometric mean

Method	Form of Score($\mathbf{x}, \mathbf{y}$)
No Reward/Normalization	Score($\mathbf{x}, \mathbf{y}$) = $\log \Pr(\mathbf{y} \mathbf{x})$
Length Reward	Score($\mathbf{x}, \mathbf{y}$) = $\log \Pr(\mathbf{y} \mathbf{x}) + \lambda \cdot n$
Bounded Length Reward	Score($\mathbf{x}, \mathbf{y}$) = $\log \Pr(\mathbf{y} \mathbf{x}) + \lambda \cdot \max(l_p, n)$
Length Normalization (Basic)	Score($\mathbf{x}, \mathbf{y}$) = $\frac{\log \Pr(\mathbf{y} \mathbf{x})}{n}$
Length Normalization (GNMT)	Score($\mathbf{x}, \mathbf{y}$) = $\frac{\log \Pr(\mathbf{y} \mathbf{x})}{(5+n)^\alpha / (5+1)^\alpha}$

Table 5.2: Scoring functions for length reward and length normalization. $m = |\mathbf{x}|$, $n = |\mathbf{y}|$, and $l_p = \text{scalar}_p \cdot m$. λ and α are parameters.

of the probabilities $\{\Pr(y_i | \mathbf{y}_{<i}, \mathbf{x})\}$ ⁴

Another prominent example is the normalization strategy adopted in the GNMT system [Wu et al., 2016]. It employs an exponentially scaled and shifted length penalty:

$$n_{\text{correct}} = \frac{(5+n)^\alpha}{(5+1)^\alpha}, \quad (5.70)$$

where the power α is a hyper-parameter and can be determined empirically on a tuning set. To compare different methods, Table 5.2 shows a list of scoring functions for length reward and length normalization.

In machine translation, the length problem is intrinsically linked to the **coverage** problem, a topic studied extensively in SMT. When translating a source sequence, it is important to monitor how often each word is translated. We consider **over-translation** to occur (yielding an erroneously long translation) if source words are translated multiple times, and **under-translation** to occur (yielding a truncated translation) if source words are insufficiently translated. Traditionally, the coverage of a source sequence is modeled using an m -dimensional **coverage vector** $\begin{bmatrix} v_1 & \dots & v_m \end{bmatrix}$. Each element v_j indicates the degree to which the source word x_j has been translated. In SMT systems, v_j is typically a binary variable: 0 denotes untranslated, and 1 denotes translated. However, standard NMT systems lack such an explicit symbolic coverage mechanism. Instead, they rely on attention weights distributed across the target words. Therefore, one way to quantify the coverage of a source word x_j is to measure how strongly it is attended to by the generated target words. To achieve this, we redefine v_j as a continuous

⁴Suppose $\{a_1, \dots, a_n\}$ are n variables. Since

$$\exp\left(\frac{\sum_{i=1}^n \log a_i}{n}\right) = \left(\prod_{i=1}^n a_i\right)^{\frac{1}{n}}, \quad (5.68)$$

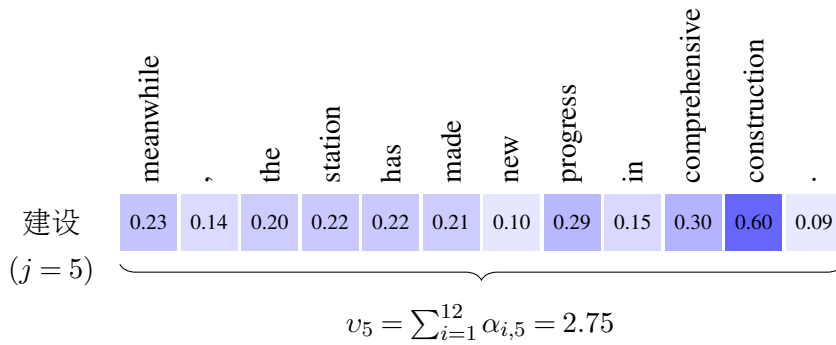
we have

$$\frac{\sum_{i=1}^n \log a_i}{n} = \log\left(\prod_{i=1}^n a_i\right)^{\frac{1}{n}}. \quad (5.69)$$

variable:

$$v_j = \sum_{i=1}^n \alpha_{i,j}. \quad (5.71)$$

v_j can thus be viewed as the “number of times” x_j is translated, e.g., $v_j = 0$ means that x_j is not translated at all, and $v_j = 1$ means that x_j is counted only once in translation. Consider the example in Figure 5.8. For the source-side word 建设, the corresponding attention weights are shown below.



We will say that 建设 is translated 2.75 times. It is possible to make use of $\{v_1, \dots, v_m\}$ to define how much the source-side sequence is covered in translation. A simple way to do this is to develop a coverage score $\text{cp}(\mathbf{x}, \mathbf{y})$ by combining $\{v_1, \dots, v_m\}$. For example, the GNMT system defines $\text{cp}(\mathbf{x}, \mathbf{y})$ in the following form

$$\text{cp}(\mathbf{x}, \mathbf{y}) = \beta \sum_{j=1}^m \log(\min(v_j, 1)), \quad (5.72)$$

where β dictates the weight of the coverage model. The intuition is that if $v_j \geq 1$, the word x_j is considered adequately translated; if $v_j < 1$, x_j lacks sufficient translation. As a result, $\text{cp}(\mathbf{x}, \mathbf{y})$ penalizes hypotheses where source words are under-translated. A refined version of this penalty was proposed by Li et al. [2018]:

$$\text{cp}(\mathbf{x}, \mathbf{y}) = \beta \sum_{j=1}^m \log(\max(v_j, \gamma)), \quad (5.73)$$

where γ is the hyper-parameter for truncation, giving a tolerance for under-translation. A similar form was proposed in [Chorowski and Jaitly, 2017]

$$\text{cp}(\mathbf{x}, \mathbf{y}) = \beta \sum_{j=1}^m \mathbb{1}(v_j > \gamma). \quad (5.74)$$

It counts the number of times that $v_j > \gamma$.

$\text{cp}(\mathbf{x}, \mathbf{y})$ can be easily introduced into search by adding it to $\text{Score}(\mathbf{x}, \mathbf{y})$. For example, the GNMT-style scoring function is defined to be

$$\text{Score}(\mathbf{x}, \mathbf{y}) = \frac{\log \Pr(\mathbf{y}|\mathbf{x})}{(5+n)^\alpha / (5+1)^\alpha} + \text{cp}(\mathbf{x}, \mathbf{y}). \quad (5.75)$$

In practice, modifying $\text{Score}(\mathbf{x}, \mathbf{y})$ is not the only method for mitigating the length and coverage problems during search. An alternative strategy involves introducing architectural modifications to explicitly model these phenomena [Tu et al., 2016; Mi et al., 2016a; Sankaran et al., 2016; See et al., 2017; Malaviya et al., 2018]. Note that in some cases, the length of the target sequence is specified or predicted in advance. In such cases, we can either employ non-autoregressive models [Gu et al., 2018] or design length-controllable text generation systems for specific applications [Rush et al., 2015; Kikuchi et al., 2016].

5.4.2 Pruning and Beam Search

There are many ways to define a search space. As a general concept in computer science, a search space is typically defined as the set of all possible solutions to a problem. For sequence-to-sequence problems, we can view a hypothesis as a mapping from a source-side sequence $\mathbf{x}$ to a target-side sequence $\mathbf{y}$, and the search space as the collection of all such hypotheses⁵.

We can implement a search algorithm by organizing hypotheses. Recall that in Eqs. (5.61) and (5.62), we compute the probability of $\mathbf{y}$ given $\mathbf{x}$ using a left-to-right factorization. A typical search system maintains a set of hypotheses (or partial hypotheses) and expands them incrementally⁶. The search procedure begins with an initial hypothesis set Z_0 containing a single hypothesis z_0 , whose target side is $y_0 = \langle \text{SOS} \rangle$, representing the start symbol for all target sequences. We then expand this hypothesis set over a series of search steps. Let $Z_0 \dots Z_{n_{\max}}$ be a sequence of hypothesis sets, where $n_{\max}$ is the maximum number of search steps. At step i , we extend each hypothesis by appending a new word v_k drawn from the vocabulary V_y . Let $z.\text{src}$ and $z.\text{tgt}$ denote the source and target sides of a hypothesis z , respectively. Clearly, $z.\text{src} = \mathbf{x}$ for any z . Given a hypothesis $z_{\text{cur}} \in Z_{i-1}$, we can extend it to generate $|V_y|$ new hypotheses, denoted as $\{z_{\text{next}}^1, \dots, z_{\text{next}}^{|V_y|}\}$:

$$\begin{aligned} \{z_{\text{next}}^1, \dots, z_{\text{next}}^{|V_y|}\} &= \text{Extend}(z_{\text{cur}}, V_y) \\ &= \bigcup_{v_k \in V_y} \text{Extend}(z_{\text{cur}}, v_k). \end{aligned} \quad (5.76)$$

Here, $\text{Extend}(z_{\text{cur}}, v_k)$ is a function that extends the input hypothesis z_{cur} with a word $v_k \in V_y$.

⁵Here we use $(\mathbf{x}, \mathbf{y})$ to denote a hypothesis. When there are multiple mappings from $\mathbf{x}$ to $\mathbf{y}$, a hypothesis can be represented as $(\mathbf{x}, \mathbf{y}, d)$ where d denotes the specific mapping. For example, if we transform $\mathbf{x}$ to $\mathbf{y}$ using a synchronous grammar, there might be multiple derivations of grammar rules to achieve this.

⁶A hypothesis is considered partial when the corresponding target-side sequence does not end with $\langle \text{EOS} \rangle$, meaning it is an incomplete sequence. In this section, we use the terms *hypothesis* and *partial hypothesis* interchangeably because their forms are identical.

The target side of the resulting hypothesis is the concatenation of $z_{\text{cur}}.tgt$ and v_k , written as⁷:

$$z_{\text{next}}^k.tgt = z_{\text{cur}}.tgt \circ v_k. \quad (5.77)$$

These new hypotheses $\{z_{\text{next}}^1, \dots, z_{\text{next}}^{|V_y|}\}$ are then added to Z_i . Figure 5.9 illustrates the first few steps of this extension process. All hypotheses can be organized into a tree structure, where Z_i corresponds to the set of nodes at level i of the search tree. We thus have:

$$|Z_i| = |V_y| \cdot |Z_{i-1}|. \quad (5.78)$$

In other words, the size of Z_i grows exponentially with the number of steps, yielding $|Z_i| = |V_y|^i$.

Each hypothesis z is associated with a log-probability $\log \Pr(z.tgt|z.src)$. Following the form of Eq. (5.62), this probability can be computed recursively:

$$\begin{aligned} \log \Pr(z_{\text{next}}^k.tgt|z_{\text{next}}^k.src) &= \log \Pr(z_{\text{cur}}.tgt|z_{\text{cur}}.src) + \\ &\quad \log \Pr(v_k|z_{\text{cur}}.tgt, z_{\text{cur}}.src). \end{aligned} \quad (5.79)$$

As an example, suppose $z_{\text{next}}^k.tgt = y_0 \dots y_{i+1}$. The form of Eq. (5.79) becomes clear from the following expression

$$\begin{aligned} \frac{\log \Pr(y_0 \dots y_{i+1}|\mathbf{x})}{\log \Pr(z_{\text{next}}^k.tgt|z_{\text{next}}^k.src)} &= \frac{\log \Pr(y_0 \dots y_i|\mathbf{x})}{\log \Pr(z_{\text{cur}}.tgt|z_{\text{cur}}.src)} + \frac{\log \Pr(y_{i+1}|y_0 \dots y_i, \mathbf{x})}{\log \Pr(v_k|z_{\text{cur}}.tgt, z_{\text{cur}}.src)} \\ &= \sum_{k=1}^i \log \Pr(y_k|\mathbf{y}_{<k}, \mathbf{x}) + \log \Pr(y_{i+1}|y_0 \dots y_i, \mathbf{x}) \\ &= \sum_{k=1}^{i+1} \log \Pr(y_k|\mathbf{y}_{<k}, \mathbf{x}). \end{aligned} \quad (5.80)$$

Given this probability, we can then compute $z.score = \text{Score}(z.src, z.tgt)$, as in Section 5.4.1. This score allows us to rank and compare different hypotheses. If a hypothesis ends with the $\langle \text{EOS} \rangle$ symbol, it is considered complete and is no longer extended. Once complete, it is pushed onto a max-heap⁸. We can then extract the highest-scoring hypotheses from this heap. The search procedure typically terminates when a predefined number of complete hypotheses are found, for instance, when the heap reaches its capacity. The resulting search algorithm is presented below.

Algorithm: A Simple Breadth-first Search Algorithm

`SimpleSearch`($\mathbf{x}$)

⁷We use $a \circ b$ to denote the concatenation of two strings a and b .

⁸Given a max-heap a , we use $a.\text{Pop}()$ to denote a function that removes and returns the top item of a , and $a.\text{PopAll}()$ for a function that extracts all items.

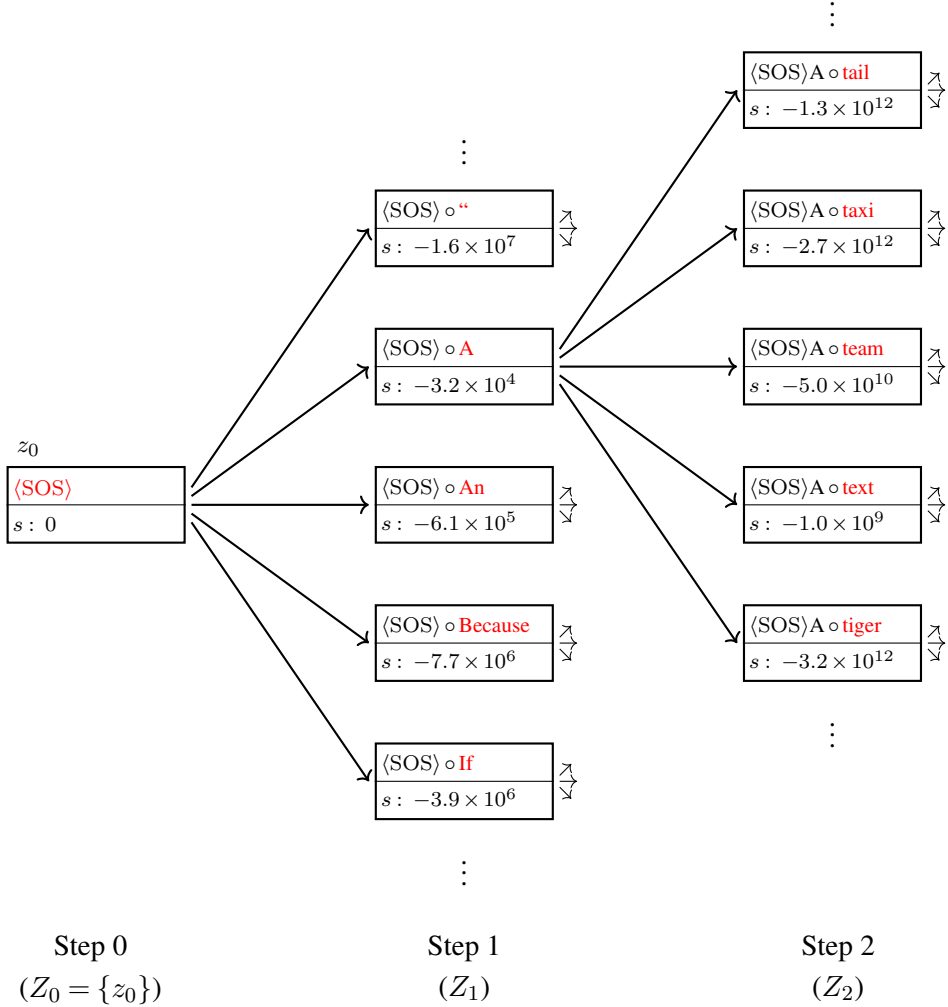


Figure 5.9: Illustration of hypothesis extension in the first 3 steps. Each (partial) hypothesis is represented as a box in which we show the corresponding target-side sequence and model score. Each search step is associated with a hypothesis set Z_i . We start with a hypothesis $z_0 \in Z_0$ denoting the start symbol $\langle \text{SOS} \rangle$. In step i , we extend every hypothesis in Z_{i-1} by trying to append every word from a vocabulary V_y (see words in red). This operation will result in $|V_y| \cdot |Z_{i-1}|$ hypotheses, forming the hypothesis set Z_i . The hypothesis extension procedure represents a breadth-first search algorithm: we create all the nodes (or search states) at depth $i - 1$ before moving to depth i . A tree structure is created along with this procedure, and a leaf node of the tree can trace the search path back to the root node.

// Search for the best hypothesis given the source-side sequence $\mathbf{x}$

1. Create a Heap with $size_{\text{heap}}$ elements
2. $Z_0 = \{z_0\}$ where $z_0.src = \mathbf{x}$ and $z_0.tgt = y_0$
3. For each step $i = 1$ to n_{max}

```

4.   For each hypothesis  $z_{\text{cur}} \in Z_{i-1}$ 
5.       For each word  $v_k \in V_y$ 
6.            $z_{\text{next}} = \text{Extend}(z_{\text{cur}}, v_k, \mathbf{x})$ 
7.           If  $z_{\text{next}}.tgt$  ends with  $\langle \text{EOS} \rangle$ , then
8.               Add  $z_{\text{next}}$  to Heap
9.           Else
10.              Add  $z_{\text{next}}$  to  $Z_i$ 
11.              If Heap is full and/or other stopping criteria are met, then
12.                  Break all the loops
13. Return Heap.Pop()
Extend( $z_{\text{cur}}, v_k, src$ )
// Create a new hypothesis by appending a new word  $v_k$  to the target-side of  $z_{\text{cur}}$ 
1.   Create a new hypothesis  $z_{\text{next}}$ 
2.    $z_{\text{next}}.src = src$ 
3.    $z_{\text{next}}.tgt = z_{\text{cur}}.tgt \circ v_k$ 
4.    $z_{\text{next}}.prob = z_{\text{cur}}.prob + \log \Pr(v_k | z_{\text{cur}}.tgt, z_{\text{cur}}.src)$  // see Eq. (5.79)
5.    $z_{\text{next}}.score = \text{Score}(z_{\text{next}}.src, z_{\text{next}}.tgt)$  // see Section 5.4.1
6.   Return  $z_{\text{next}}$ 

```

If the hypothesis heap has infinite capacity ($size_{\text{heap}} = \infty$), this algorithm performs an exhaustive search over the space of all hypotheses with lengths up to n_{max} , producing at most $1 + |V_y| + |V_y|^2 + \dots + |V_y|^{n_{\text{max}}} = \frac{|V_y|^{n_{\text{max}}+1} - 1}{|V_y| - 1}$ complete hypotheses. This search space is extremely large and computationally intractable in practice⁹. As a result, practical systems heavily rely on pruning techniques to render the search process feasible. In the remainder of this section, we will introduce two popular search algorithms that utilize pruning for efficient decoding.

1. Greedy Search

The **greedy strategy** is a fundamental concept in algorithm design. It is based on the heuristic that a globally optimal solution can be obtained by making locally optimal decisions. Although this strategy only guarantees local optimality, it is sufficient for many practical applications, and its low computational complexity is a clear advantage.

Applying the greedy strategy to our search problem is straightforward. At each extension step i , we only consider the single best hypothesis up to this step. To be more precise, for any set Z_i , we retain only the hypothesis with the highest model score and discard the rest. The best hypothesis at each step of the greedy search is given by

$$z_{\text{best}} = \arg \max_{z_{\text{next}} \in \text{Extend}(Z_{i-1}, V_y)} z_{\text{next}}.score. \quad (5.81)$$

Here the function $\text{Extend}(Z_{i-1}, V_y)$ has the same meaning as that in Eq. (5.76), but operates

⁹Consider, for example, a vocabulary size of 20,000 ($|V_y| = 20,000$) and a length limit of 20 ($n_{\text{max}} = 20$). The total number of hypotheses $\frac{|V_y|^{n_{\text{max}}+1} - 1}{|V_y| - 1}$ would be approximately 1.05×10^{86} .

on a set of hypotheses, that is,

$$\text{Extend}(Z_{i-1}, V_y) = \bigcup_{z \in Z_{i-1}} \text{Extend}(z, V_y). \quad (5.82)$$

Then, Z_i is defined to be

$$Z_i = \{z_{\text{best}}\}. \quad (5.83)$$

A greedy search algorithm for sequence-to-sequence problems is given below.

Algorithm: A Greedy Search Algorithm

GreedySearch($\mathbf{x}$)

// Search for the “best” hypothesis in a greedy manner

1. Create a hypothesis z_{best}
2. $Z_0 = \{z_0\}$ where $z_0.\text{src} = \mathbf{x}$ and $z_0.\text{tgt} = y_0$
3. For each step $i = 1$ to n_{max}
4. $z_{\text{best}}.\text{score} = -\infty$
5. For each hypothesis $z_{\text{cur}} \in Z_{i-1}$
6. For each word $v_k \in V_y$
7. $z_{\text{next}} = \text{Extend}(z_{\text{cur}}, v_k, \mathbf{x})$
8. If $z_{\text{best}}.\text{score} < z_{\text{next}}.\text{score}$, then
9. $z_{\text{best}} = z_{\text{next}}$
10. If $z_{\text{best}}.\text{tgt}$ ends with $\langle \text{EOS} \rangle$ and/or other stopping criteria are met, then
11. Break the loop
12. $Z_i = \{z_{\text{best}}\}$
13. Return z_{best}

At each search step, we have only one active hypothesis to extend (i.e., $|Z_{i-1}| = 1$). Thus, we generate $|V_y|$ extensions, from which we select the best one for the subsequent step. The total number of times $\text{Extend}(z_{\text{cur}}, v_k)$ is called is $|V_y| \cdot n_{\text{max}}$. Assuming $\text{Extend}(z_{\text{cur}}, v_k)$ operates in constant time, the overall time complexity of the algorithm scales linearly with respect to $|V_y|$ and n_{max} .

2. Beam Search

Beam search is a natural extension of the above greedy search algorithm. It is based on the greedy heuristic as well, and is thus a type of greedy algorithm. The core idea of beam search is to retain a predetermined number of the most promising hypotheses at each step, rather than just the single best one. A beam is a data structure that stores the best hypotheses we have generated so far. The number of hypotheses in a beam is a predetermined parameter, called **beam width** or **beam size**. Here we can simply view Z_i as a beam, written as

$$Z_i = \{z_{\text{best}}^1, \dots, z_{\text{best}}^{\text{size}_{\text{beam}}}\}, \quad (5.84)$$

where $size_{beam}$ is the beam size. z_{best}^1 is the best hypothesis in the set $Extend(Z_{i-1}, V_y)$ (see Eq. (5.81)), z_{best}^2 is the 2nd best hypothesis in $Extend(Z_{i-1}, V_y)$, and so on.

The following pseudo-code describes a beam search algorithm for sequence-to-sequence problems.

Algorithm: A Beam Search Algorithm

BeamSearch($\mathbf{x}$)

// Search for the “best” hypothesis by considering a number of best candidates

// in each step

1. Create a Heap with $size_{heap}$ elements
2. $Z_0 = \{z_0\}$ where $z_0.src = \mathbf{x}$ and $z_0.tgt = y_0$
3. For each step $i = 1$ to n_{max}
4. Create a heap Beam with $size_{beam}$ elements
5. For each hypothesis $z_{cur} \in Z_{i-1}$
6. For each word $v_k \in V_y$
7. $z_{next} = \text{Extend}(z_{cur}, v_k, \mathbf{x})$
8. If $z_{next}.tgt$ ends with $\langle \text{EOS} \rangle$, then
9. Add z_{next} to Heap
10. Else
11. **UpdateBeam**(Beam, z_{next})
12. If Heap is full and/or other stopping criteria are met, then
13. Break all the loops
14. $Z_i = \text{Beam.PopAll}()$
15. Return $\text{Heap.Pop}()$

UpdateBeam(Beam, z_{next})

// Update Beam with a newly-generated hypothesis z_{next}

1. Add z_{next} to Beam ^a

^aBeam is a bounded max-heap that maintains the top- $size_{beam}$ hypotheses. So, if $z_{next}.score$ is lower than the lowest score in the heap, it is discarded. In other words, Beam only stores top- $size_{beam}$ best hypotheses and ignores the rest.

The function **UpdateBeam**(Beam, z_{next}) can be viewed as an instance of **histogram pruning**. Note that this general-purpose framework provides a simple way to implement other pruning methods, and one can modify **UpdateBeam**(Beam, z_{next}) as needed. For example, an alternative method, called **threshold pruning**, retains the hypotheses whose model scores differ from the best hypothesis in Beam by less than a predefined threshold, e.g., we discard z_{next} in **UpdateBeam**(Beam, z_{next}) if

$$z_{next}.score < z_{best}.score - \theta_{beam}, \quad (5.85)$$

where z_{best} is the best hypothesis in Beam. Alternatively, we can consider a relative threshold

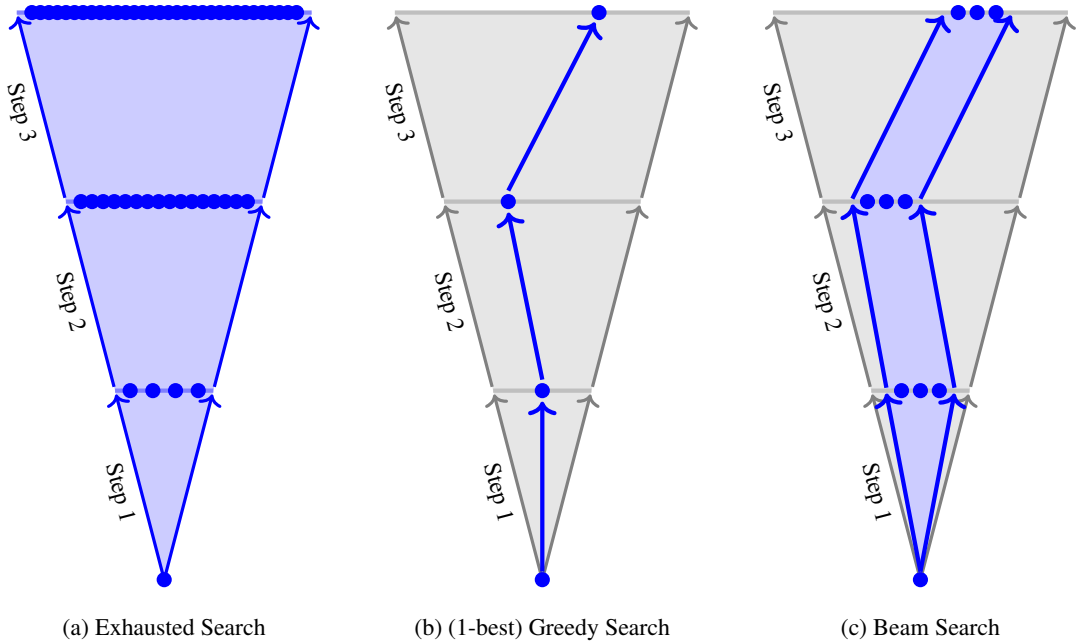


Figure 5.10: A comparison of exhaustive search, (1-best) greedy search and beam search. Balls represent search states or partial hypotheses. Exhaustive search explores all search states in the search space. By contrast, greedy search keeps only the 1-best path of search states and prunes away the rest. Beam search is a trade-off between them and keeps the most promising search states in a beam in each step.

method [Freitag and Al-Onaizan, 2017], given by

$$z_{\text{next}} \cdot \text{score} < z_{\text{best}} \cdot \text{score} \cdot \theta_{\text{beam}}. \quad (5.86)$$

Figure 5.10 shows a comparison of exhaustive search, (1-best) greedy search and beam search. At one extreme, the optimal solution is guaranteed, but an exponentially large number of search states are visited. At the other extreme, only the minimum number of search states are visited, but the solution is sub-optimal. By contrast, beam search makes a trade-off between the two methods. A larger beam size means more search effort and a greater chance of finding better hypotheses, while a smaller beam size means faster search and a higher risk of missing the optimum. It is also possible to use a variable beam size to make a better trade-off during search [Buckman et al., 2016; Post and Vilar, 2018; Kulikov et al., 2019].

An important problem related to these search algorithms is the problem of **search errors**. In general, search errors can be defined in several different ways. Here we say that a search error occurs if the search result is not the same as that of exhaustive search. Intuitively, minimizing search errors should yield higher-quality results. Thus, one might expect that increasing the beam size would consistently improves the generated sequences. However, this assumption does not always hold true. For instance, in neural machine translation systems, decoding with an excessively large beam size can lead to degraded translation quality [Koehn and

[Knowles, 2017](#)]. This counterintuitive phenomenon has inspired research into the performance deterioration associated with large beam sizes [[Ott et al., 2018b](#); [Yang et al., 2018b](#); [Stahlberg and Byrne, 2019](#)].

3. Stopping Criteria

Although the time complexities of the previously discussed algorithms are bounded by the maximum number of search steps (i.e., $n_{\max}$), it is important to design more effective stopping criteria to terminate the search as early as possible, particularly for latency-sensitive applications. This typically requires heuristics to design additional criteria that halt the search procedure at an optimal point. Common stopping criteria include:

- If a given number of complete hypotheses are created, then we stop searching. For example, in the beam search algorithm described in this subsection, the search program terminates when we have $size_{\text{heap}}$ complete hypotheses. Another way to implement this idea is to shrink the beam as the number of complete hypotheses increases. In [Bahdanau et al. \[2014\]](#)’s system, once a new complete hypothesis is created, the beam size decreases by 1. Therefore, the search program will terminate if the beam size is reduced to 0.
- If every hypothesis at step i has a score lower than that of the best complete hypothesis in Heap by some margin, then we stop searching. Suppose $z_{\text{bestinall}}$ is the best hypothesis we have generated so far (i.e., $z_{\text{bestinall}} = \text{Heap.Pop}()$). If every hypothesis z_{next} at step i satisfies

$$z_{\text{bestinall}}.\text{score} - z_{\text{next}}.\text{score} \geq \theta_{\text{all}}, \quad (5.87)$$

then the search process concludes. Here, θ_{all} is a hyper-parameter typically determined empirically. A simple choice is $\theta_{\text{all}} = 0$, which is employed in several popular sequence-to-sequence frameworks [[Ott et al., 2019](#)]. Under certain conditions, such an **early stopping** strategy can even guarantee search optimality [[Huang et al., 2017](#); [Yang et al., 2018b](#)].

- If every hypothesis at step i has a score lower than that of the last complete hypothesis in Heap by some margin, then we stop searching. This is a weak condition for early stopping.
- If the top-ranked hypotheses at step i are all complete hypotheses, then we stop searching. This is a more aggressive version of early stopping. For example, in [Klein et al. \[2017\]](#)’s system, the search program terminates at step i if the top-1 hypothesis is a complete hypothesis.
- If the computational budget (e.g., a maximum number of floating-point operations or a strict wall-clock time limit) is exhausted, the search is terminated. In applications where computational resources are highly constrained and latency is a primary concern, this criterion becomes essential.

Occasionally, the search algorithm may fail to find any complete hypothesis before hitting the length limit $n_{\max}$. In practice, a common fallback strategy in such cases is to complete the

highest-scoring partial hypothesis by appending the $\langle \text{EOS} \rangle$ token to its end.

Note that selecting appropriate stopping criteria reflects a compromise between computational efficiency and predictive accuracy during inference (often referred to as the **speed-accuracy trade-off**). While allocating more computational time does not guarantee superior outputs for a sequence-to-sequence system, it is important to understand how closely our heuristic results approximate the optimal solution by exploring a broader region of the search space. Exact search algorithms that guarantee such optimality are discussed further in Section 5.4.4.

5.4.3 Online Search

So far in our general discussion of sequence-to-sequence problems, we have assumed that all the source-side words are observed as a complete sequence. However, in some practical applications, the inputs are received sequentially, and we wish to make predictions conditioned on partial observations. An example of this is online automatic speech recognition in which the system continually takes new acoustic signals and simultaneously outputs the corresponding transcription units.

Intuitively, we might think of the generation of the i -th target-side word as a problem of mapping a prefix of the source-side sequence to a target-side word. We can formulate this by introducing a function $g(i)$ which denotes the maximum length of the prefix of $\mathbf{x}$ we use in generating y_i . Thus, the probability of y_i given the entire sequence $\mathbf{x}$ and the previously generated words $\mathbf{y}_{<i}$ can be approximated by

$$\Pr(y_i | \mathbf{y}_{<i}, \mathbf{x}) \approx \Pr(y_i | \mathbf{y}_{<i}, \mathbf{x}_{\leq g(i)}), \quad (5.88)$$

where $\mathbf{x}_{\leq g(i)}$ denotes the sub-sequence $x_1 \dots x_{g(i)}$. Then, the log probability of the target-side sequence $\mathbf{y}$ given the source-side sequence $\mathbf{x}$ is written as

$$\begin{aligned} \log \Pr(\mathbf{y} | \mathbf{x}) &= \sum_{i=1}^n \log \Pr(y_i | \mathbf{y}_{<i}, \mathbf{x}) \\ &\approx \sum_{i=1}^n \log \Pr(y_i | \mathbf{y}_{<i}, \mathbf{x}_{\leq g(i)}). \end{aligned} \quad (5.89)$$

This equation frames a sequence-to-sequence problem as a prefix-to-prefix problem, that is, the prefix $\mathbf{y}_{\leq i}$ depends only on the prefix $\mathbf{x}_{\leq g(i)}$. Inference for this model is simple. For each i , the search system waits until the first $g(i)$ source-side words have been received, and then extends the hypotheses as usual. This can be done by reusing the algorithms described in the previous subsection. For example, we can modify the beam search algorithm and obtain the following online search algorithm.

Algorithm: An Online Beam Search Algorithm

`OnlineBeamSearch( $\mathbf{x}, g(\cdot)$ )`

// Online search executed once a sufficient number of input words are received.

```

// At each search step, a predefined number of promising candidates are considered.
1.  Create a Heap with capacity  $size_{heap}$ 
2.   $Z_0 = \{z_0\}$  where  $z_0.tgt = y_0$ 
3.   $j = 1$  // source word index
4.   $i = 1$  // target generation step
5.   $input = \emptyset$ 
6.  While  $i \leq n_{max}$  do
7.    If  $j \leq g(i)$ , then // read a word from the input stream
8.       $input = input \circ x_j$ 
9.       $j++$ 
10.   Else // make a prediction at step  $i$ 
11.     // when  $g(i)$  input words are observed (stored in  $input$ )
12.     Create a bounded heap Beam of size  $size_{beam}$ 
13.     For each hypothesis  $z_{cur} \in Z_{i-1}$ 
14.       For each word  $v_k \in V_y$ 
15.          $z_{next} = \text{Extend}(z_{cur}, v_k, input)$ 
16.         If  $input$  equals  $\mathbf{x}$  and  $z_{next}.tgt$  ends with  $\langle \text{EOS} \rangle$ , then
17.           Add  $z_{next}$  to Heap
18.         Else
19.            $\text{UpdateBeam}(\text{Beam}, z_{next})$ 
20.         If Heap is full and/or other stopping criteria are met, then
21.           Break all the loops
22.        $\text{OutputPartial}(\text{Beam})$ 
23.      $Z_i = \text{Beam.PopAll}()$ 
24.      $i++$ 
25. Return  $\text{Heap.Pop}()$ 
 $\text{OutputPartial}(\text{Beam})$ 
// Output a partial result
1.  Display the best hypothesis in Beam

```

An advantage of this system is that the output at step i can be produced immediately once we have seen $\mathbf{x}_{\leq g(i)}$. This results in an **online sequence-to-sequence system** in which input words arrive in a continuous stream and predictions are generated as soon as a sufficient number of input words have been observed.

While the search problem here seems simple, much remains to be done to define $g(i)$. Clearly, $g(i)$ is a monotonically non-decreasing function. As a simple example, we can define $g(i) = m$ for any i . This reduces the above algorithm exactly to the standard beam search algorithm that works with a complete input sequence. By contrast, in online sequence-to-sequence tasks, we want $g(i)$ to be as small as possible, and so we can start computation as early as possible in inference. The simplest case is that the input and output sequences are synchronous in some way. For example, an automatic speech recognition system assigns each spectral frame a transcription unit. In this case, we have a simple correspondence between

inputs and outputs: $m = n$ (i.e., $|\mathbf{x}| = |\mathbf{y}|$), and x_i corresponds to y_i . Then, we can simply define $g(i) = i$, in other words, each time a new input arrives, we make a prediction.

A more complicated case is online sequence-to-sequence problems with reordering, such as **simultaneous translation**, in which a target-side word may depend on source-side words with long-range dependencies. A simple way to address this is to delay the predictions for a number of steps. For example, the wait- k method forces each prediction to lag behind the inputs by k words [Ma et al., 2019]. More formally, the wait- k version of the function $g(i)$ is defined to be

$$g(i) = \min(m, k + i - 1). \quad (5.90)$$

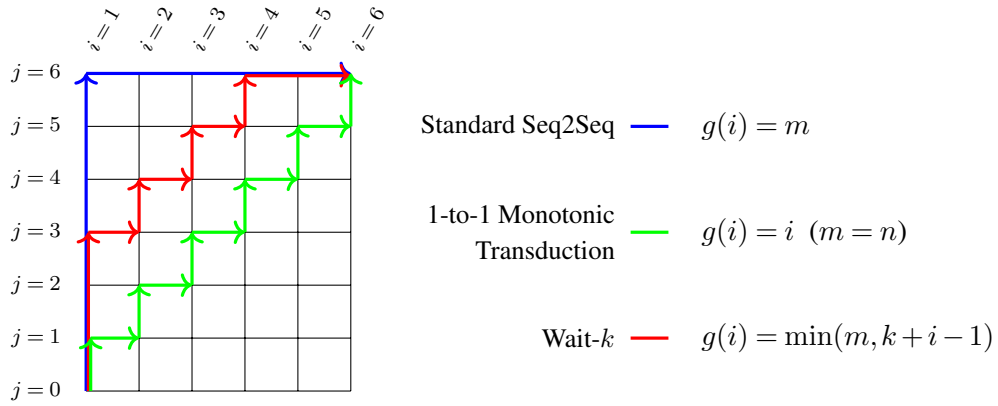
Here k is a hyper-parameter that controls how large a source-side context is considered in predicting target-side words. When $k = \infty$, it is the same as the standard search methods for sequence-to-sequence inference. In simultaneous translation and related tasks, results are in general satisfactory by using a small value of k . A comparison of different $g(i)$ is shown in Figure 5.11.

In some applications of online sequence-to-sequence problems, we may know when to perform search and when to read inputs. For example, in **interactive machine translation** [Casacuberta et al., 2009], the translation of a partial input sequence is triggered directly by user actions (e.g., key presses). In this case, we do not need to define $g(i)$, but view it as an input variable of the model.

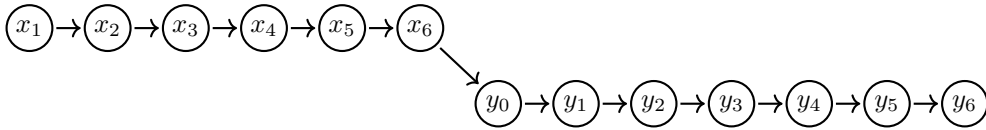
Note that while one can directly employ pre-trained sequence-level models for online inference, developing such systems often requires additional training effort. A more principled approach to online sequence-to-sequence modeling is to model the transformation from $\mathbf{x}$ to $\mathbf{y}$ as a sequence of actions [Grissom II et al., 2014; Cho and Esipova, 2016; Gu et al., 2017; Zheng et al., 2019]. For example, an action can be either a predict operation that performs search at the current step, or a read operation that accepts a new input word. Then, we can frame the task of designing the function $g(i)$ as learning a policy to determine which action is taken given a source-side prefix $\mathbf{x}_{\leq j}$ and a target-side prefix $\mathbf{y}_{< i}$. Thus, sequence-to-sequence models can be trained on the states of these action sequences so that they can make better predictions conditioned on part of the input. However, a discussion of training online sequence-to-sequence models lies outside the scope of this section. We refer the reader to the above papers for more details on these methods.

5.4.4 Exact Search

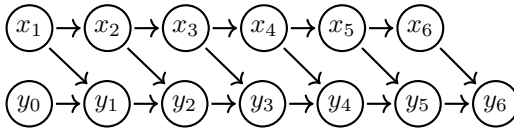
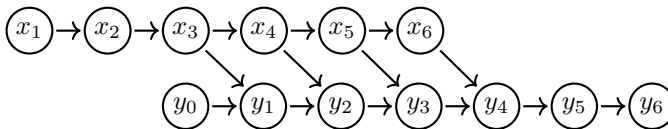
From a formal point of view, we would ideally like to develop a system with no search errors. Although approximate search algorithms have been used successfully in many applications, it is important to study model errors of these systems, and thus to focus on the problem in principle, not just in practice. So developing exact search algorithms for sequence-to-sequence models has long been an interesting topic in NLP research. However, the search problem for a simple word-based machine translation system with n -gram language models has been found to be an NP-hard problem [Knight, 1999]. Much of the earlier research formulated the search problem

(a) Visualization of $g(i)$.

Standard Seq2Seq



1-to-1 Monotonic Transduction

Wait- k ($k = 3$)

(b) Action sequences.

Figure 5.11: Visualization (top) and action sequences (bottom) of different $g(i)$ for a pair of sequences ($\mathbf{x} = x_1 \dots x_6, \mathbf{y} = y_1 \dots y_6$). In an action sequence, a circled x_j stands for the action of reading a source-side word (x_j), and a circled y_i stands for the action of predicting the probability of y_i given $\mathbf{x}_{\leq g(i)}$ and $\mathbf{y}_{< i}$. Arrows here stand for dependencies between words. Because y_0 denotes the start symbol $\langle \text{SOS} \rangle$, it could be generated without dependencies on any words.

as classical **combinatorial optimization problems**, such as the linear programming problem and the traveling salesman problem, and employed the corresponding solvers [Germann et al., 2004; Zaslavskiy et al., 2009]. Additional research efforts explored exact search algorithms for

statistical machine translation systems by using the Lagrangian relaxation technique [Chang and Collins, 2011; Rush and Collins, 2012] and finite-state automata [de Gispert et al., 2010; Allauzen et al., 2014].

Unlike these methods, which are more or less dependent on the integration of n -gram language models into sequence-to-sequence models, the models described in this chapter take a simpler form. We begin with a basic model in which the scoring function $\text{Score}(\mathbf{x}, \mathbf{y})$ is the log probability $\log \Pr(\mathbf{y}|\mathbf{x})$. Eq. (5.62) tells us that $\log \Pr(\mathbf{y}|\mathbf{x})$ can be written as a sum of word-level log probabilities, and $\log \Pr(\mathbf{y}|\mathbf{x})$ becomes smaller as more target-side words are generated (i.e., a larger n)¹⁰. In other words, $\log \Pr(\mathbf{y}|\mathbf{x})$ is a monotonically decreasing function with respect to the target-side length n : for any i , we have

$$\begin{aligned} \log \Pr(\mathbf{y}_{\leq i}|\mathbf{x}) &= \log \Pr(\mathbf{y}_{< i}|\mathbf{x}) + \log \Pr(y_i|\mathbf{y}_{< i}, \mathbf{x}) \\ &\leq \log \Pr(\mathbf{y}_{< i}|\mathbf{x}). \end{aligned} \quad (5.91)$$

This is also called the **monotonicity** of the scoring function.

Then, by making use of the monotonic nature of model scores, we can develop a heuristic to rule out hypotheses that would never be the best. Let $z_{\text{bestinall}}$ be the globally best complete hypothesis discovered so far. If a new partial hypothesis has a model score strictly lower than $z_{\text{bestinall}}.\text{score}$, it cannot possibly be extended to surpass it, and thus we do not need to extend it. This allows us to explore a region that is significantly smaller than the original search space without any loss of optimality. Note that as the search proceeds, $z_{\text{bestinall}}.\text{score}$ continually increases. It becomes increasingly difficult to find a better hypothesis, resulting in more aggressive pruning as the search continues. The pseudo-code below outlines an exact search algorithm for the sequence-to-sequence model defined by Eq. (5.62).

Algorithm: An Exact Search Algorithm

ExactSearch($\mathbf{x}$)

// Search for the “best” hypothesis by making use of the monotonicity of the
// scoring function ($\text{Score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x})$).

1. Create a priority queue (max-heap) Queue
2. Create a hypothesis z_{best} with $z_{\text{best}}.\text{score} = -\infty$
3. While Queue is not empty do
4. $z_{\text{cur}} = \text{Queue.Pop}()$
5. If $|z_{\text{cur}}.\text{tgt}| > n_{\text{max}}$, then
6. skip z_{cur} and continue the loop
7. For each word $v_k \in V_y$
8. $z_{\text{next}} = \text{Extend}(z_{\text{cur}}, v_k, \mathbf{x})$
9. $\text{bound} = z_{\text{best}}.\text{score}$ // a lower bound on model scores
10. If $\text{bound} < z_{\text{next}}.\text{score}$, then // admissible pruning

¹⁰Consider $\log \Pr(\mathbf{y}|\mathbf{x}) = \sum_{i=1}^n \log \Pr(y_i|\mathbf{y}_{< i}, \mathbf{x})$. Since $\log \Pr(y_i|\mathbf{y}_{< i}, \mathbf{x})$ has a non-positive value, $\log \Pr(\mathbf{y}|\mathbf{x})$ will be smaller or unchanged if n grows.

11. If $z_{\text{next}}.tgt$ ends with $\langle \text{EOS} \rangle$, then
12. $z_{\text{best}} = z_{\text{next}}$
13. $bound = z_{\text{next}}.score$
14. Else
15. Add z_{next} to Queue
16. Return z_{best}

This is a general algorithm for exact search, and its search efficiency is greatly influenced by the design of the priority queue [Meister et al., 2020]. For example, we can view $\text{Score}(\mathbf{x}, \mathbf{y})$ as the priority of each hypothesis in the priority queue, as in a max-heap¹¹. Then, the resulting algorithm performs a procedure of breadth-first-like search, since a hypothesis with a shorter target-side sequence is more likely to have a higher model score and to be a top-ranked item in the priority queue. For efficient search, however, we wish to find complete hypotheses as early as possible, such that more unpromising hypotheses can be thrown away in the early stage of search. To do this, we can bias the priority of a hypothesis towards a longer target-side sequence. This provides a **depth-first search** algorithm which is more likely to find complete hypotheses in a shorter time [Stahlberg and Byrne, 2019].

While the exact search algorithm becomes apparent by considering the monotonicity of $\Pr(\mathbf{y}|\mathbf{x})$, in practical systems, as discussed in Section 5.4.1, $\text{Score}(\mathbf{x}, \mathbf{y})$ often has a more complex form involving length reward or normalization, and so the monotonic property does not hold. Fortunately, the assumption of monotonicity can be dropped at the expense of slightly relaxing the lower bound on model scores for pruning. Here we define *bound* to be the lowest model score that a hypothesis should have so that it can at best be extended to a hypothesis as good as z_{best} . For example, consider a simple word reward model described in Eq. (5.63): $\text{Score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda \cdot n$. For a hypothesis z_{next} , there are at most $n_{\text{max}} - |z_{\text{next}}.tgt|$ words we can predict to obtain a complete hypothesis. Suppose all these $n_{\text{max}} - |z_{\text{next}}.tgt|$ words are predicted with a probability of 1. Then, the model score of the resulting hypothesis (denoted by z_{new}) will be given by

$$\begin{aligned}
 z_{\text{new}}.score &= z_{\text{next}}.score + \sum_{i=|z_{\text{next}}.tgt|+1}^{n_{\text{max}}} (\log 1 + \lambda) \\
 &= z_{\text{next}}.score + \lambda \cdot (n_{\text{max}} - |z_{\text{next}}.tgt|).
 \end{aligned} \tag{5.92}$$

Using this result, we can define *bound* as

$$bound = z_{\text{best}}.score - \lambda \cdot (n_{\text{max}} - |z_{\text{next}}.tgt|). \tag{5.93}$$

An alternative way to derive the lower bound is to simply subtract the maximum possible

¹¹We can implement a priority queue using a max-heap.

word reward, given by

$$bound = z_{\text{best}} \cdot score - \lambda \cdot n_{\text{max}}. \quad (5.94)$$

This is a loose lower bound and leads to less pruning.

In the case of length normalization, we can do this in a similar way. For example, consider the length normalization model $\text{Score}(\mathbf{x}, \mathbf{y}) = \frac{\log \Pr(\mathbf{y}|\mathbf{x})}{n}$, as in Eqs. (5.66-5.67). A lower bound on admissible model scores is given by

$$bound = \frac{\Pr(z_{\text{next}} \cdot tgt|\mathbf{x})}{n_{\text{max}}}. \quad (5.95)$$

In practice, such a lower bound can be defined in several different ways to guarantee the optimality of search, depending on which model and search strategy are used in the sequence-to-sequence systems [Huang et al., 2017; Stahlberg and Byrne, 2019].

We can easily apply these lower bounds to the above exact search algorithm by replacing line 9 with Eq. (5.93) or (5.95). As a side effect, the search will explore more hypotheses and thus be much slower.

5.4.5 Differentiable Search

We have addressed the search problem through the introduction of heuristic search algorithms in which we try to maximize the scoring function on a set of sequences of discrete variables. An alternative is to relax these discrete variables to continuous variables and formulate the problem as a **continuous optimization** problem [Hoang et al., 2017; Kumar et al., 2021]. While we try to use a consistent notation throughout this book, it is convenient here to introduce notation that slightly differs from that used in previous chapters. We will use a vector $\mathbf{y}_i^w \in \{0, 1\}^{|V_y|}$ to denote the one-hot representation of y_i . Suppose the output at step i is a distribution over V_y , denoted by $\Pr(\cdot|\mathbf{y}_{<i}, \mathbf{x})$. Then, we can express the log probability of y_i as a dot product of two vectors:

$$\begin{aligned} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) &= \mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_{<i}, \mathbf{x}) \\ &= \mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x}), \end{aligned} \quad (5.96)$$

where $\mathbf{y}_{<i} = y_0 \dots y_{i-1}$ is represented as a sequence of one-hot vectors $\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w$. As discussed in Chapter 3, the right-hand side of the above equation corresponds to selecting the entry y_i of the vector $\log \Pr(\cdot|\mathbf{y}_{<i}, \mathbf{x})$ (or $\log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x})$).

Using this notation, we can write $\log \Pr(\mathbf{y}|\mathbf{x})$ as

$$\begin{aligned} \log \Pr(\mathbf{y}|\mathbf{x}) &= \sum_{i=1}^n \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \\ &= \sum_{i=1}^n \mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x}). \end{aligned} \quad (5.97)$$

Provided we use $\log \Pr(\mathbf{y}|\mathbf{x})$ as the objective function (i.e., $\text{Score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x})$), the search problem can be formulated as

$$\hat{\mathbf{y}}_0^w \dots \hat{\mathbf{y}}_n^w = \arg \max_{\mathbf{y}_1^w \dots \mathbf{y}_n^w} \sum_{i=1}^n \mathbf{y}_i^w \cdot \log \Pr(\cdot | \mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x}). \quad (5.98)$$

This is equivalent to the standard form for inference of sequence-to-sequence models, given by

$$\begin{aligned} \hat{\mathbf{y}} &= \hat{y}_0 \dots \hat{y}_n \\ &= \arg \max_{y_0 \dots y_n} \Pr(y_0 \dots y_n | \mathbf{x}). \end{aligned} \quad (5.99)$$

Given Eq. (5.98), we can now relax each one-hot vector to a real-valued vector with a constraint that the sum of all its entries is equal to 1, that is,

$$\mathbf{y}_i^w \in \mathbb{R}_+^{|V_y|} \quad (5.100)$$

$$\text{s.t. } \|\mathbf{y}_i^w\|_1 = 1. \quad (5.101)$$

In this way, $\mathbf{y}_i^w$ can be informally treated as a $|V_y|$ -dimensional representation of y_i , although it possesses many more dimensions than typical dense embeddings used in NLP. Instead of corresponding to a specific word in the vocabulary, $\mathbf{y}_i^w$ now describes a continuous distribution over the vocabulary. As a result, the term $\mathbf{y}_i^w \cdot \log \Pr(\cdot | \mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x})$ computes the **expected log-probability** of the prediction at step i . To make the conditional distribution $\Pr(\cdot | \mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x})$ differentiable with respect to the continuous prefix, [Hoang et al. \[2017\]](#) replace the standard discrete word embedding lookup at each decoder step with an **expected embedding**, which is a weighted average of the word embedding matrix guided by the probability vector $\mathbf{y}_{i-1}^w$. Interestingly, this formulation implies that Eq. (5.98) defines a “new” task in which we attempt to maximize a continuous objective function (i.e., a sum of n expected log-probabilities).

We can solve Eq. (5.98) using off-the-shelf optimization toolkits. Because $\mathbf{y}_i^w$ is constrained to a **simplex**¹², it is straightforward to apply general **constrained optimization** algorithms. Alternatively, one can use algorithms that are designed to solve the optimization problem with simplex constraints.

A third approach to solving Eq. (5.98) is to explicitly incorporate the constraints into the

¹²A simplex is a concept from geometry. In a Euclidean space, a k -simplex is a k -dimensional polytope described by a set of $k+1$ independent points $\{\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_k\}$. This polytope is defined as the set of points

$$P_{k\text{-simplex}} = \{a_0 \cdot \mathbf{p}_0 + a_1 \cdot \mathbf{p}_1 + \dots + a_k \cdot \mathbf{p}_k\}, \quad (5.102)$$

where

$$\sum_{i=0}^k a_i = 1, \quad (5.103)$$

$$a_i \geq 0 \text{ for all } i \in [0, k]. \quad (5.104)$$

objective function and to use gradient descent methods to optimize this function. For example, [Hoang et al. \[2017\]](#) modify Eq. (5.98) and obtain a new form for optimization

$$\hat{\mathbf{y}}_0^w \dots \hat{\mathbf{y}}_n^w = \arg \max_{\mathbf{y}_1^w \dots \mathbf{y}_n^w} \sum_{i=1}^n \text{Softmax}(\mathbf{y}_i^w) \cdot \log \Pr(\cdot | \mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x}). \quad (5.105)$$

Here we remove the simplex constraint on $\mathbf{y}_i^w$, and instead impose it via the Softmax output.

Once we have obtained the optimal sequence $\hat{\mathbf{y}}_0^w \dots \hat{\mathbf{y}}_n^w$, we need to map each $\mathbf{y}_i^w$ to a word. A straightforward method is to select the word corresponding to the entry of $\mathbf{y}_i^w$ with the largest value. However, this may break the optimality of the solution because the condition $\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w$ is changed when these variables are discretized. A more practical method is to perform optimization to predict the next word given a prefix, e.g., we fix $\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w$ to the one-hot representations of the optimal prefix, and maximize $\sum_{k=i}^n \mathbf{y}_k^w \cdot \log \Pr(\cdot | \mathbf{y}_0^w \dots \mathbf{y}_{k-1}^w, \mathbf{x})$. Then, we select the best word at position i and move on to the next position.

So far, we have assumed that the search objective is derived from the log probability $\log \Pr(\mathbf{y} | \mathbf{x})$ and that the output length is predetermined. To search over sequences of varying lengths, we can repeat the above optimization procedure for every $n \in [1, n_{\max}]$ and select the sequence with the highest score. This approach also facilitates the integration of length normalization and rewards into the search. Specifically, we can ignore length bias during the fixed- n optimization and apply length models post-hoc. That is, we leave the initial search objective unchanged and, in the final step, select the best candidate across all lengths based on a length-normalized $\text{Score}(\mathbf{x}, \mathbf{y})$.

5.4.6 Hypothesis Diversity

Generating multiple outputs is often necessary when one wants to rescore these outputs and/or interact with the system. One of the most widely used methods is to employ beam search to generate top-ranked hypotheses. For example, we can simply view the elements of Heap as the k -best hypotheses in beam search (see Section 5.4.2). However, this approach often produces highly similar hypotheses, and it is difficult to figure out which one is better even when more options are available. Table 5.3 shows the 4-best outputs of a text summarization system. We see that these texts are fairly similar to each other. One reason for this phenomenon is that diverse hypotheses, even those that might eventually achieve high model scores upon completion, will be pruned if they fall out of the beam at any intermediate stage of the search. From a modeling perspective, we can interpret this as an inherent limitation of the locally normalized models used here: every prediction is made at an intermediate step of the search, and there is no mechanism to correct past decisions once a prefix is committed [[Murray and Chiang, 2018](#)].

One approach to improving hypothesis diversity is to introduce penalties when the hypotheses in the beam lack variation [[Li and Jurafsky, 2016](#); [Vijayakumar et al., 2018](#)]. A simple objective function is given by

$$\text{Score}_d(\mathbf{x}, \mathbf{y}) = \text{Score}(\mathbf{x}, \mathbf{y}) - \lambda \cdot dp. \quad (5.106)$$

Rank	Output
1	Manuela Arbelaez accidentally revealed the correct answer to a guessing game for a new Hyundai Sonata. Host Drew Carey couldn't stop laughing. It's been a busy week for "The Price Is Right" when Bob Barker, 91, showed up to run his old show.
2	Manuela Arbelaez accidentally revealed the correct answer to a guessing game for a new Hyundai Sonata. Host Drew Carey couldn't stop laughing. It's been a busy week for "The Price Is Right" when Bob Barker showed up to run his old show.
3	Manuela Arbelaez accidentally revealed the correct answer to a guessing game for a new Hyundai Sonata. Host Drew Carey couldn't stop laughing. It's been a busy week for "The Price Is Right" when Bob Barker, 91, showed up to run the show.
4	Manuela Arbelaez accidentally revealed the correct answer to a guessing game for a new Hyundai Sonata. Host Drew Carey couldn't stop laughing. It's been a busy week for "The Price Is Right" when Bob Barker, 91, showed up to run his show.

Table 5.3: 4-best outputs of a text summarization system on a sample in the CNN/Daily Mail dataset (beam size = 4). We see that these texts differ only by a few words.

It combines the original model score $\text{Score}(\mathbf{x}, \mathbf{y})$ and a diversity penalty dp . dp can be defined in several ways. One approach is to penalize hypotheses that are close in the search tree. For example, one can define dp as the rank of a hypothesis in the set of its siblings that are extended from the same parent hypothesis, and so the beam can spread its members over a larger region of the hypothesis space [Li and Jurafsky, 2016]. Another way to introduce diversity measures is to consider the differences among the target-side sequences of the hypotheses in the beam. For example, we can define dp as the average string similarity between a given hypothesis and other hypotheses in the beam [Xiao et al., 2013].

The above idea can also be expressed as constraints imposed on the search procedure. For example, we can constrain the beam to include only the hypotheses that are rooted at different parents in the last step [Boulanger-Lewandowski et al., 2013]. More precisely, for each hypothesis $z_{\text{cur}} \in Z_{i-1}$, we select the best next-step hypothesis as follows

$$\hat{z}_{\text{next}} = \arg \max_{z_{\text{next}} \in \text{Extend}(z_{\text{cur}}, V_y)} \Pr(z_{\text{next}}.tgt | \mathbf{x}). \quad (5.107)$$

The hypothesis $\hat{z}_{\text{next}}$ is then added to Z_i . Note that this is essentially a **subspace method** that divides the hypothesis space into subspaces, and collects results over these subspaces. An intuition behind this method is that different subspaces can describe different aspects of the problem, and so we can have diverse solutions.

Another approach to addressing the diversity issue is to perturb beam search by introducing randomly generated hypotheses into the beam [Holtzman et al., 2020; Wiher et al., 2022]. One common way to do this is to choose some random words for extending a hypothesis, and to add the extended hypotheses to the beam. In general, these words can be sampled from the distribution $\Pr(\cdot | \mathbf{y}_{<i}, \mathbf{x})$ over the entire vocabulary or its subset. Randomness can also be

added to the system inputs at test time. For example, an input word can be represented as a linear combination of its original embedding and the embedding of a word sampled from a randomly selected training sequence [Li et al., 2021]. In problems having many local minima, adding random noise to the search process is generally helpful, as we can explore more diverse hypotheses and prevent the system from getting stuck in certain regions of the search space.

Instead of performing search using a single system, we can use multiple systems to obtain diverse hypotheses. These systems can be built on either different architectures or different hidden structures/configurations [He et al., 2018; Shen et al., 2019; Wu et al., 2020; Sun et al., 2020]. Although such methods do not fall under the search framework that we have been discussing, combining the results from multiple systems is generally helpful. The following section will present a discussion on this issue.

5.4.7 Combining Multiple Models

In machine learning, **ensembling** refers to methods aimed at addressing modeling limitations rather than search issues. In this subsection, we discuss these methods because their implementations typically require modifications to the search modules, and we can gain some insight into the resulting system by viewing it from the search perspective.

Ensemble methods aim to improve predictive performance by combining the outputs of multiple **constituent systems** (or **component systems**). The problem of combining multiple models has been extensively discussed since the emergence of statistical models in NLP, where such techniques are often referred to as **system combination methods** to emphasize their practical utility. For the sequence-to-sequence models discussed here, a widely adopted form of system combination is prediction averaging [Sutskever et al., 2014]. Suppose we have an ensemble of K trained sequence-to-sequence models. The log-probability of generating the target-side word y_i , given its left context $\mathbf{y}_{<i}$ and the source-side sequence $\mathbf{x}$, can be defined using the geometric average:

$$\log \Pr(y_i | \mathbf{y}_{<i}, \mathbf{x}) = \frac{1}{K} \sum_{k=1}^K \log \Pr_k(y_i | \mathbf{y}_{<i}, \mathbf{x}), \quad (5.108)$$

or alternatively by using the arithmetic average

$$\log \Pr(y_i | \mathbf{y}_{<i}, \mathbf{x}) = \log \frac{1}{K} \sum_{k=1}^K \Pr_k(y_i | \mathbf{y}_{<i}, \mathbf{x}), \quad (5.109)$$

where $\Pr_k(y_i | \mathbf{y}_{<i}, \mathbf{x})$ is the output of the k -th component system. These forms are so simple that one can implement them for any sequence-to-sequence models without significant modifications to existing systems, and they have been used as the basis of many successful systems in various evaluation tasks [Barrault et al., 2020; Akhbardeh et al., 2021].

A limitation of prediction averaging is that all component systems must adhere to the same basic autoregressive formulation (see Eq. (5.62)), and we need direct access to their

step-wise output probabilities $\{\Pr_k(y_i|y_{<i}, \mathbf{x})\}$. When dealing with black-box systems, we must instead perform sequence-level ensembling. A common strategy is to aggregate the complete sequences produced by the component systems. The most straightforward approach is **hypothesis selection** [Hildebrand and Vogel, 2008], where we simply select the single “best” sequence from the generated pool based on a predefined criterion. An alternative is to regenerate a novel sequence that may differ from any individual output in the pool [Matusov et al., 2006; Rosti et al., 2007]. This typically requires compressing the generated sequences into a compact unified structure (such as a word lattice), followed by an additional search pass (e.g., lattice decoding and rescoring) to extract the optimal consensus sequence [Deoras et al., 2011; Stahlberg et al., 2016; Khayrallah et al., 2017].

Note that the ensembling of sequence-to-sequence models is related to the diversity issue discussed in the previous subsection. It is widely recognized that component systems must exhibit sufficient diversity to yield meaningful ensemble gains. Thus, models should be trained or constructed to capture different behavioral patterns [Sutskever et al., 2014; Zhou et al., 2017]. One of the most popular methods is **checkpoint ensembling**. It uses multiple snapshots of a single model at different checkpoints during the training process and combines their predictions via averaging. This method can be useful for alleviating the overfitting problem in practice. Also, different models can be created from a base model under different settings. For example, we can build models with different numbers of parameters based on a common backbone model. A more general approach is to take models based on different architectures, although this is at the expense of more development effort.

Another way to view sequence ensembling is that it provides a two-pass search scheme. In the first pass, multiple independent systems perform inference. Because each system possesses its own inductive biases regarding modeling and decoding, they naturally explore different regions of the search space. A high-quality hypothesis discovered by one system might be entirely missed by another. The outcome of this first pass is a diverse pool of hypotheses, each considered “optimal” from a specific perspective. In the second pass, this diverse pool defines a constrained, high-quality search space, over which a fine-grained rescoring model can navigate to determine the final, globally optimal result.

5.4.8 More Search Objectives

In this subsection, we consider more objective functions that can be applied to the search problem.

1. Search with Future Scores

Most of the algorithms described previously can be viewed as variants of best-first search algorithms [Meister et al., 2020]. A* search is another example of a best-first search algorithm and is widely considered to be a good solution to the general search problem. Vanilla A* search requires that all search states are sorted at every search step, which is computationally intractable in many NLP problems. We therefore continue to focus on beam search and greedy search, but employ an A*-like objective function instead. Specifically, given a search state

$(\mathbf{x}, \mathbf{y}_{\leq i})$, the A^* -like objective function can be defined as:

$$\text{Score}_{A^*}(\mathbf{x}, \mathbf{y}_{\leq i}) = g(\mathbf{x}, \mathbf{y}_{\leq i}) + h(\mathbf{x}, \mathbf{y}_{\leq i}). \quad (5.110)$$

Here $g(\mathbf{x}, \mathbf{y}_{\leq i})$ is the cumulative reward from the start state to $(\mathbf{x}, \mathbf{y}_{\leq i})$, and $h(\mathbf{x}, \mathbf{y}_{\leq i})$ is the estimated reward of the “optimal” path from $(\mathbf{x}, \mathbf{y}_{\leq i})$ to the final goal. Because $g(\mathbf{x}, \mathbf{y}_{\leq i})$ and $h(\mathbf{x}, \mathbf{y}_{\leq i})$ can take arbitrary forms, this framework is very general. For example, if we define

$$g(\mathbf{x}, \mathbf{y}_{\leq i}) = \text{Score}(\mathbf{x}, \mathbf{y}_{\leq i}), \quad (5.111)$$

$$h(\mathbf{x}, \mathbf{y}_{\leq i}) = 0, \quad (5.112)$$

then $\text{Score}_{A^*}(\mathbf{x}, \mathbf{y})$ reduces to the objective functions discussed previously.

To make full use of this formulation, it is natural to seek a function that estimates future rewards or costs. Ideally, we would want $h(\mathbf{x}, \mathbf{y}_{\leq i})$ to exactly compute how much additional reward can be obtained by extending $(\mathbf{x}, \mathbf{y}_{\leq i})$ to the optimal complete hypothesis. This, however, is intractable because it requires exploring all possible hypotheses extended from $(\mathbf{x}, \mathbf{y}_{\leq i})$ to identify the best one. In practice, it is common to use a computationally cheaper model that approximates the true future reward. Conventional approaches rely on heuristics to define $h(\mathbf{x}, \mathbf{y}_{\leq i})$ [Koehn et al., 2007], such as estimating the contributions of words that may be generated. These heuristics can be adapted based on the architectural design of sequence-to-sequence models [He et al., 2017; Zheng et al., 2018]. A more principled approach is to use a value-based formulation [Ren et al., 2017; Li et al., 2017; Leblond et al., 2021]. One can develop a policy that predicts the distribution of y_i given $\mathbf{x}$ and $\mathbf{y}_{< i}$, alongside a **value function** that estimates future rewards. Eq. (5.110) can therefore be interpreted as a linear combination of the policy score for $(\mathbf{x}, \mathbf{y}_{\leq i})$ and its corresponding value. Such a treatment of search objectives falls into the framework of **value-based search** and has been successfully employed in reinforcement learning [Silver et al., 2017].

2. Search with Language Models

For a long time, language models have played a central role in text generation tasks. For example, statistical machine translation and automatic speech recognition systems typically rely on large n -gram language models to produce fluent text. While modern sequence-to-sequence models do not require separate language models, integrating them during sequence-to-sequence search remains intuitively appealing for machine translation and related problems.

Following the convention that a language model can be treated as a feature of a log-linear (or linear) model [Och and Ney, 2002], the language model-augmented objective can be defined as

$$\text{Score}_{\text{lm}}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda \cdot \log \Pr(\mathbf{y}). \quad (5.113)$$

This formulation does not involve length reward and normalization terms, but both can be easily incorporated as additional features. In general, the language model $\Pr(\mathbf{y})$ is trained solely on target-side sequences, enabling the use of large-scale monolingual data in sequence-

to-sequence models [Gulcehre et al., 2017]. Interestingly, it has been found that current sequence-to-sequence models are themselves strong language models if they are sufficiently trained, and a more effective strategy for leveraging target-side data is to create synthetic corpora, a technique known as **data augmentation**. An example is **back-translation**, where a target-to-source system translates monolingual target sentences to generate synthetic source sentences. This synthetic bilingual data is then used to augment the training set [Sennrich et al., 2016; Edunov et al., 2018]. In many tasks, such a simple method can achieve significant improvements in translation quality, but these results raise questions about the necessity of using additional language models in neural machine translation.

Note that the model of Eq. (5.113) depends on the choice of the coefficient λ . For machine translation, we are usually interested in a positive value of λ so that our system can produce more fluent outputs. By contrast, a negative value of λ means that we want less frequent outputs. For example, if $\lambda = -1$, then Eq. (5.113) can be written as the point-wise mutual information of $\mathbf{x}$ and $\mathbf{y}$

$$\begin{aligned} \text{Score}_{\text{lm}}(\mathbf{x}, \mathbf{y}) &= \log \Pr(\mathbf{y}|\mathbf{x}) - \log \Pr(\mathbf{y}) \\ &= \log \frac{\Pr(\mathbf{x}, \mathbf{y})}{\Pr(\mathbf{x}) \cdot \Pr(\mathbf{y})}. \end{aligned} \quad (5.114)$$

This scoring function has been shown to be useful for generating more diverse outputs for neural conversation systems [Li et al., 2016].

3. Minimum Bayes Risk Search

So far, our discussion of search objectives has focused on selecting the highest-scoring hypothesis, a process known as **maximum a posteriori (MAP)** search¹³. An underlying assumption

¹³In statistics, MAP is a method for inferring the parameters of a statistical model. Suppose we have a model that describes the distribution of a variable x , parameterized by θ . MAP seeks the optimal value of θ by maximizing the probability of θ given x , written as:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} \Pr(\theta|x). \quad (5.115)$$

$\hat{\theta}_{\text{MAP}}$ is also called the **mode** of the posterior distribution of θ . For the MAP search problem here, we simply denote θ by $\mathbf{y}$ and seek the mode of $\Pr(\mathbf{y}|\mathbf{x})$.

As a Bayesian method, we can re-express the above equation using Bayes' rule

$$\begin{aligned} \hat{\theta}_{\text{MAP}} &= \arg \max_{\theta} \frac{\Pr(x|\theta) \cdot \Pr(\theta)}{\Pr(x)} \\ &= \arg \max_{\theta} \Pr(x|\theta) \cdot \Pr(\theta), \end{aligned} \quad (5.116)$$

where θ is treated as a random variable with a prior distribution $\Pr(\theta)$.

By contrast, MLE directly maximizes the likelihood function $\Pr(x|\theta)$

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \Pr(x|\theta). \quad (5.117)$$

Thus, the MAP result can be viewed as an estimate of θ that considers both the likelihood of x given θ and the prior distribution of θ . Note that MAP and MLE are equivalent if $\Pr(\theta)$ is a uniform distribution.

of this method is that the posterior probability $\Pr(\mathbf{y}|\mathbf{x})$ (or the model score $\text{Score}(\mathbf{x}, \mathbf{y})$) correlates with the quality of the outputs. In practice, this assumption offers several useful properties. For instance, the search system is straightforward to implement, and the search objective remains consistent with the training objective. However, MAP search suffers from several shortcomings, which have prompted researchers to explore more powerful alternatives. One issue with MAP search is that its objective does not align with the actual evaluation metrics of the system. The metrics used in the end-to-end evaluation of a system may take forms that differ significantly from $\Pr(\mathbf{y}|\mathbf{x})$. A second problem is that MAP, as a specific point estimate in Bayesian decision theory, provides no measure of uncertainty and is often overconfident. In certain applications, sequence-to-sequence models spread probability mass too thinly across many different hypotheses [Ott et al., 2018a], meaning the MAP estimate may fail to capture the bulk of the true distribution.

Here, we consider **minimum Bayes risk (MBR)** search, which provides a principled way to incorporate evaluation metrics into the decoding process and leverage the full distribution over hypotheses. The MBR method defines a risk function over a pair of sequences, denoted by $R(\mathbf{y}, \mathbf{y}_r)$. It computes the cost of predicting $\mathbf{y}$ when the true reference is $\mathbf{y}_r$, evaluated according to a specific metric. For example, in machine translation, we could define the risk score as $1 - \text{BLEU}$. The total risk for a candidate $\mathbf{y}$ over a set Ω is then defined as the expectation of $R(\mathbf{y}, \mathbf{y}_r)$ with respect to the posterior distribution $\Pr(\mathbf{y}_r|\mathbf{x})$:

$$\begin{aligned} \text{Risk}(\mathbf{y}) &= \mathbb{E}_{\mathbf{y}_r \sim \Pr(\mathbf{y}_r|\mathbf{x})} R(\mathbf{y}, \mathbf{y}_r) \\ &= \sum_{\mathbf{y}_r \in \Omega} R(\mathbf{y}, \mathbf{y}_r) \cdot \Pr(\mathbf{y}_r|\mathbf{x}). \end{aligned} \quad (5.118)$$

However, the summation over all possible target-side sequences is computationally infeasible. We therefore restrict Ω to the k -best outputs or sampled outputs of a system [Eikema and Aziz, 2020], denoted by Ω_{system} . Then, we take $\text{Score}(\mathbf{x}, \mathbf{y}) = -\text{Risk}(\mathbf{y})$ and obtain the following objective for MBR search

$$\begin{aligned} \hat{\mathbf{y}} &= \arg \max_{\mathbf{y}} -\text{Risk}(\mathbf{y}) \\ &= \arg \min_{\mathbf{y}} \sum_{\mathbf{y}_r \in \Omega_{\text{system}}} R(\mathbf{y}, \mathbf{y}_r) \cdot \Pr(\mathbf{y}_r|\mathbf{x}). \end{aligned} \quad (5.119)$$

This model is very general and applies to a wide range of NLP problems in which one needs to search for an optimal hypothesis in a large set of candidates [Goodman, 1996; Goel and Byrne, 2000; Kumar and Byrne, 2004]. It allows for flexible forms of risk functions, for instance, incorporating various factors in evaluating hypotheses. MBR search has recently been of interest to NLP researchers because it has been found to be effective in eliminating the biases caused by MAP search [Müller and Sennrich, 2021; Freitag et al., 2022]. In addition to providing a formulation of search objectives, MBR methods can be used for training sequence-to-sequence models, and are considered a promising solution to the discrepancy between training and evaluation objectives [Shen et al., 2016].

5.5 Summary

In this chapter, we provided an overview of sequence-to-sequence modeling, which serves as a foundation for many modern NLP systems. Sequence-to-sequence modeling is a very rich area of research, and has been widely discussed in different disciplines, even beyond NLP. This chapter is not a review of all the literature on this subject, but focuses on some of the core methods and ideas. We started with an introduction to sequence-to-sequence problems, as well as the encoder-decoder architecture which lays the foundations for most of the state-of-the-art sequence-to-sequence systems. As an illustration of the application of this architecture, we considered the problem of neural machine translation, and built a simple neural machine translation model using the basic knowledge we have learned so far.

We also presented the attention mechanism and a series of refinements. If we look back to the past few years, we find that exploring attention models is the next natural step in developing sequence-to-sequence models. Although attention models were initially popularized by their success in machine translation, they have since come to dominate the NLP community. There is also great interest in attention models in some other sub-fields of AI, such as computer vision [Borji and Itti, 2012; Xu et al., 2015; Jaderberg et al., 2015] and speech processing [Chorowski et al., 2015; Chan et al., 2016; Bahdanau et al., 2016]. As a result, the past few years have been an exciting time for people in these areas.

Sequence-to-sequence models are so successful that there is a growing trend toward unifying diverse problems under a single framework. Not only have we developed powerful sequence-to-sequence models to deal with very general problems, but current research trends increasingly favor unified approaches. An example is the Transformer, a self-attention-based sequence-to-sequence model that has become a fundamental backbone for tasks spanning various data modalities from text to vision and speech. It can even be extended to deal with multimodal problems which are sometimes more challenging. This makes things more interesting and exciting: an improvement to one model can be used to improve systems in a variety of tasks. And we are seeing a significant change in our research paradigm in which the NLP and machine learning fields are converging, and results in NLP research are becoming more influential. However, on the other side of the coin is that we are allocating more attention to some problems while neglecting others. In recent NLP conferences, there are many papers discussing how to train big sequence-to-sequence models and apply them to different text generation tasks, but there are a relatively small number of papers on parsing. There have always been debates on this over the past few decades, for example, what and how much prior knowledge do we need to build an NLP system? [Church, 2011; See, 2018] A detailed discussion of these debates is beyond the scope of this chapter. Fortunately, NLP research promises to continue to be diverse and active, providing valuable insights from both perspectives. For example, there are interesting findings that the neural sequence models can learn some linguistic properties from data, and linguistic structures can help system design. In Chapter 6, we will see a few examples.

The “bias” of research focus also exists on the machine learning side of problem-solving. For example, for sequence-to-sequence problems discussed here, recent years have witnessed

a drastic increase of interest in model design and training methods, but only a relatively small group of people discuss the search problem. While search is a classical problem in AI and plays an important role in practical systems [Russell and Norvig, 2010], it is not even discussed in recent tutorials and surveys in NLP. This motivates us to write a section on this subject so that we can have a more complete picture of the problem. However, our general discussion does not cover all aspects of the search problem. One important topic we left out is efficiency [Birch et al., 2018; Heafield et al., 2021]. While this chapter includes some discussions on the efficiency issue, such as stopping criteria of search algorithms, efficient methods are a wide-ranging topic and are generally dependent on model architectures. A more detailed discussion of them can be found in Chapter 6. Another topic that one may be interested in is **constrained search** in which constraints are imposed on the search process [Hokamp and Liu, 2017; Anderson et al., 2017]. In general, these constraints come from our prior knowledge or interactions with users. For example, constrained search has been used to enforce term translation constraints on machine translation [Hasler et al., 2018; Post and Vilar, 2018].

One last note on limitations of this chapter. The formulation of the general sequence-to-sequence problem described here is based on the left-to-right factorization of $\Pr(y|x)$, resulting in an autoregressive model. One limitation of this formulation is that each prediction at some step depends only on the preceding words, and so the model cannot access the right context. To make use of the right context of a word, a simple approach is to build another model that performs right-to-left generation. The left-to-right and right-to-left models can then be combined to generate a better output sequence [Liu et al., 2016a; Hoang et al., 2017; Zhang et al., 2018; 2020a]. An alternative approach is given by **non-autoregressive generation** or **non-autoregressive decoding** in which the constraint of autoregressive generation is removed and each word prediction is conditioned on the global context [Gu et al., 2018; Ghazvininejad et al., 2019; Lee et al., 2020]. A key advantage of non-autoregressive generation is the possibility of system speed-up, since all the words in a sequence can be generated in parallel and we can do this efficiently using GPUs.

Bibliography

- [Akhbardeh et al., 2021] Farhad Akhbardeh, Arkady Arkhangorodsky, Magdalena Biesialska, Ondřej Bojar, Rajen Chatterjee, Vishrav Chaudhary, Marta R. Costa-jussa, Cristina España-Bonet, Angela Fan, Christian Federmann, Markus Freitag, Yvette Graham, Roman Grundkiewicz, Barry Haddow, Leonie Harter, Kenneth Heafield, Christopher Homan, Matthias Huck, Kwabena Amponsah-Kaakyire, Jungo Kasai, Daniel Khashabi, Kevin Knight, Tom Kocmi, Philipp Koehn, Nicholas Lourie, Christof Monz, Makoto Morishita, Masaaki Nagata, Ajay Nagesh, Toshiaki Nakazawa, Matteo Negri, Santanu Pal, Allahsera Auguste Tapo, Marco Turchi, Valentin Vydrin, and Marcos Zampieri. Findings of the 2021 conference on machine translation (WMT21). In *Proceedings of the Sixth Conference on Machine Translation*, pages 1–88, 2021.
- [Allauzen et al., 2014] Cyril Allauzen, Bill Byrne, Adrià de Gispert, Gonzalo Iglesias, and Michael Riley. Pushdown automata in statistical machine translation. *Computational Linguistics*, 40(3): 687–723, 2014.
- [Anderson et al., 2017] Peter Anderson, Basura Fernando, Mark Johnson, and Stephen Gould. Guided open vocabulary image captioning with constrained beam search. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 936–945, 2017.
- [Bahdanau et al., 2014] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.
- [Bahdanau et al., 2016] Dzmitry Bahdanau, Jan Chorowski, Dmitriy Serdyuk, Philemon Brakel, and Yoshua Bengio. End-to-end attention-based large vocabulary speech recognition. In *2016 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pages 4945–4949. IEEE, 2016.
- [Barrault et al., 2020] Loïc Barrault, Magdalena Biesialska, Ondřej Bojar, Marta R. Costa-jussà, Christian Federmann, Yvette Graham, Roman Grundkiewicz, Barry Haddow, Matthias Huck, Eric Joanis, Tom Kocmi, Philipp Koehn, Chi-kiu Lo, Nikola Ljubešić, Christof Monz, Makoto Morishita, Masaaki Nagata, Toshiaki Nakazawa, Santanu Pal, Matt Post, and Marcos Zampieri. Findings of the 2020 conference on machine translation (WMT20). In *Proceedings of the Fifth Conference on Machine Translation*, pages 1–55, 2020.
- [Birch et al., 2018] Alexandra Birch, Andrew Finch, Minh-Thang Luong, Graham Neubig, and Yusuke Oda. Findings of the second workshop on neural machine translation and generation. In *Proceedings of the 2nd Workshop on Neural Machine Translation and Generation*, pages 1–10, 2018.
- [Borji and Itti, 2012] Ali Borji and Laurent Itti. State-of-the-art in visual attention modeling. *IEEE transactions on pattern analysis and machine intelligence*, 35(1):185–207, 2012.
- [Boulanger-Lewandowski et al., 2013] Nicolas Boulanger-Lewandowski, Yoshua Bengio, and Pascal Vincent. Audio chord recognition with recurrent neural networks. In *Proceedings of 14th*

- International Society for Music Information Retrieval Conference*, 2013.
- [Brown et al., 1993] Peter F. Brown, Stephen A. Della Pietra, Vincent J. Della Pietra, and Robert L. Mercer. The mathematics of statistical machine translation: Parameter estimation. *Computational Linguistics*, 19(2):263–311, 1993.
- [Buckman et al., 2016] Jacob Buckman, Miguel Ballesteros, and Chris Dyer. Transition-based dependency parsing with heuristic backtracking. In *Proceedings of the 2016 Conference on empirical methods in natural language processing*, pages 2313–2318, 2016.
- [Casacuberta et al., 2009] Francisco Casacuberta, Jorge Civera, Elsa Cubel, Antonio L Lagarda, Guy Lapalme, Elliott Macklovitch, and Enrique Vidal. Human interaction for high-quality machine translation. *Communications of the ACM*, 52(10):135–138, 2009.
- [Chan et al., 2016] William Chan, Navdeep Jaitly, Quoc Le, and Oriol Vinyals. Listen, attend and spell: A neural network for large vocabulary conversational speech recognition. In *2016 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pages 4960–4964. IEEE, 2016.
- [Chang, 1967] Wing-Tsit Chang. *Reflections on things at hand*. Columbia University Press, 1967.
- [Chang and Collins, 2011] Yin-Wen Chang and Michael Collins. Exact decoding of phrase-based translation models through lagrangian relaxation. In *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*, pages 26–37, 2011.
- [Chaudhari et al., 2021] Sneha Chaudhari, Varun Mithal, Gungor Polatkan, and Rohan Ramanath. An attentive survey of attention models. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 12(5):1–32, 2021.
- [Chen et al., 2018] Kehai Chen, Rui Wang, Masao Utiyama, Eiichiro Sumita, and Tiejun Zhao. Syntax-directed attention for neural machine translation. In *Proceedings of the AAAI conference on artificial intelligence*, 2018.
- [Chen et al., 2020] Yen-Chun Chen, Linjie Li, Licheng Yu, Ahmed El Kholy, Faisal Ahmed, Zhe Gan, Yu Cheng, and Jingjing Liu. Uniter: Universal image-text representation learning. In *Proceedings of European conference on computer vision*, pages 104–120, 2020.
- [Chiang, 2005] David Chiang. A hierarchical phrase-based model for statistical machine translation. In *Proceedings of the 43rd annual meeting of the association for computational linguistics (acl’05)*, pages 263–270, 2005.
- [Chiang, 2007] David Chiang. Hierarchical phrase-based translation. *computational linguistics*, 33(2): 201–228, 2007.
- [Chiu and Raffel, 2018] Chung-Cheng Chiu and Colin Raffel. Monotonic chunkwise attention. In *Proceedings of the 8th International Conference on Learning Representations ICLR*, 2018.
- [Cho and Esipova, 2016] Kyunghyun Cho and Masha Esipova. Can neural machine translation do simultaneous translation? *arXiv preprint arXiv:1606.02012*, 2016.
- [Cho et al., 2014] Kyunghyun Cho, Bart van Merriënboer, Çağlar Guülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.
- [Chorowski and Jaitly, 2017] Jan Chorowski and Navdeep Jaitly. Towards better decoding and language model integration in sequence to sequence models. *Proc. Interspeech 2017*, pages 523–527, 2017.

- [Chorowski et al., 2015] Jan K Chorowski, Dzmitry Bahdanau, Dmitriy Serdyuk, Kyunghyun Cho, and Yoshua Bengio. Attention-based models for speech recognition. *Advances in neural information processing systems*, 28, 2015.
- [Church, 2011] Kenneth Church. A pendulum swung too far. *Linguistic Issues in Language Technology*, 6, 2011.
- [Cohn et al., 2016] Trevor Cohn, Cong Duy Vu Hoang, Ekaterina Vymolova, Kaisheng Yao, Chris Dyer, and Gholamreza Haffari. Incorporating structural alignment biases into an attentional neural translation model. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 876–885, 2016.
- [de Gispert et al., 2010] Adrià de Gispert, Gonzalo Iglesias, Graeme Blackwood, Eduardo R. Banga, and William Byrne. Hierarchical phrase-based translation with weighted finite-state transducers and shallow-n grammars. *Computational linguistics*, 36(3):505–533, 2010.
- [Deoras et al., 2011] Anoop Deoras, Tomáš Mikolov, and Kenneth Church. A fast re-scoring strategy to capture long-distance dependencies. In *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*, pages 1116–1127, 2011.
- [Dyer et al., 2013] Chris Dyer, Victor Chahuneau, and Noah A Smith. A simple, fast, and effective reparameterization of ibm model 2. In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 644–648, 2013.
- [Edunov et al., 2018] Sergey Edunov, Myle Ott, Michael Auli, and David Grangier. Understanding back-translation at scale. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 489–500, 2018.
- [Eikema and Aziz, 2020] Bryan Eikema and Wilker Aziz. Is map decoding all you need? the inadequacy of the mode in neural machine translation. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 4506–4520, 2020.
- [Feng et al., 2016] Shi Feng, Shujie Liu, Nan Yang, Mu Li, Ming Zhou, and Kenny Zhu. Improving attention modeling with implicit distortion and fertility for machine translation. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 3082–3092, 2016.
- [Freitag and Al-Onaizan, 2017] Markus Freitag and Yaser Al-Onaizan. Beam search strategies for neural machine translation. In *Proceedings of the First Workshop on Neural Machine Translation*, pages 56–60, 2017.
- [Freitag et al., 2022] Markus Freitag, David Grangier, Qijun Tan, and Bowen Liang. High quality rather than high model probability: Minimum bayes risk decoding with neural metrics. *Transactions of the Association for Computational Linguistics*, 10:811–825, 2022.
- [Garg et al., 2019] Sarthak Garg, Stephan Peitz, Udhyakumar Nallasamy, and Matthias Paulik. Jointly learning to align and translate with transformer models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4453–4462, 2019.
- [Germann et al., 2004] Ulrich Germann, Michael Jahr, Kevin Knight, Daniel Marcu, and Kenji Yamada. Fast and optimal decoding for machine translation. *Artificial Intelligence*, 154(1-2):127–143, 2004.
- [Ghazvininejad et al., 2019] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer.

- Mask-predict: Parallel decoding of conditional masked language models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 6112–6121, 2019.
- [Goel and Byrne, 2000] Vaibhava Goel and William J Byrne. Minimum bayes-risk automatic speech recognition. *Computer Speech & Language*, 14(2):115–135, 2000.
- [Goodman, 1996] Joshua Goodman. Parsing algorithms and metrics. In *34th Annual Meeting of the Association for Computational Linguistics*, pages 177–183, 1996.
- [Graves et al., 2014] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [Grissom II et al., 2014] Alvin Grissom II, He He, Jordan Boyd-Graber, John Morgan, and Hal Daumé III. Don’t until the final verb wait: Reinforcement learning for simultaneous machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1342–1352, 2014.
- [Gu et al., 2017] Jiatao Gu, Graham Neubig, Kyunghyun Cho, and Victor O.K. Li. Learning to translate in real-time with neural machine translation. In *Proceedings of the European Chapter of the Association for Computational Linguistics (EACL) Conference, 2017*, 2017.
- [Gu et al., 2018] Jiatao Gu, James Bradbury, Caiming Xiong, Victor O.K. Li, and Richard Socher. Non-autoregressive neural machine translation. In *Proceedings of International Conference on Learning Representations*, 2018.
- [Gulcehre et al., 2017] Caglar Gulcehre, Orhan Firat, Kelvin Xu, Kyunghyun Cho, and Yoshua Bengio. On integrating a language model into neural machine translation. *Computer Speech & Language*, 45: 137–148, 2017.
- [Guo et al., 2019] Maosheng Guo, Yu Zhang, and Ting Liu. Gaussian transformer: a lightweight approach for natural language inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 6489–6496, 2019.
- [Hasler et al., 2018] Eva Hasler, Adrià de Gispert, Gonzalo Iglesias, and Bill Byrne. Neural machine translation decoding with terminology constraints. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 506–512, 2018.
- [He et al., 2017] Di He, Hanqing Lu, Yingce Xia, Tao Qin, Liwei Wang, and Tie-Yan Liu. Decoding with value networks for neural machine translation. *Advances in Neural Information Processing Systems*, 30, 2017.
- [He et al., 2016] Wei He, Zhongjun He, Hua Wu, and Haifeng Wang. Improved neural machine translation with smt features. In *Proceedings of the Thirtieth AAAI conference on artificial intelligence*, 2016.
- [He et al., 2018] Xuanli He, Gholamreza Haffari, and Mohammad Norouzi. Sequence to sequence mixture model for diverse machine translation. In *Proceedings of the 22nd Conference on Computational Natural Language Learning*, pages 583–592, 2018.
- [Heafield et al., 2021] Kenneth Heafield, Qianqian Zhu, and Roman Grundkiewicz. Findings of the WMT 2021 shared task on efficient translation. In *Proceedings of the Sixth Conference on Machine Translation*, pages 639–651, 2021.
- [Hildebrand and Vogel, 2008] Almut Silja Hildebrand and Stephan Vogel. Combination of machine

- translation systems via hypothesis selection from combined n-best lists. In *Proceedings of the 8th Conference of the Association for Machine Translation in the Americas: Student Research Workshop*, pages 254–261, 2008.
- [Hoang et al., 2017] Cong Duy Vu Hoang, Gholamreza Haffari, and Trevor Cohn. Towards decoding as continuous optimisation in neural machine translation. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 146–156, 2017.
- [Hokamp and Liu, 2017] Chris Hokamp and Qun Liu. Lexically constrained decoding for sequence generation using grid beam search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1535–1546, 2017.
- [Holtzman et al., 2020] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *Proceedings of the 6th International Conference on Learning Representations ICLR*, 2020.
- [Huang et al., 2017] Liang Huang, Kai Zhao, and Mingbo Ma. When to finish? optimal beam search for neural text generation (modulo beam size). In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2134–2139, 2017.
- [Jaderberg et al., 2015] Max Jaderberg, Karen Simonyan, Andrew Zisserman, and koray kavukcuoglu. Spatial transformer networks. *Advances in neural information processing systems*, 28, 2015.
- [Jean et al., 2015] Sébastien Jean, Orhan Firat, Kyunghyun Cho, Roland Memisevic, and Yoshua Bengio. Montreal neural machine translation systems for wmt’15. In *Proceedings of the tenth workshop on statistical machine translation*, pages 134–140, 2015.
- [Khayrallah et al., 2017] Huda Khayrallah, Gaurav Kumar, Kevin Duh, Matt Post, and Philipp Koehn. Neural lattice search for domain adaptation in machine translation. In *Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 20–25, 2017.
- [Kikuchi et al., 2016] Yuta Kikuchi, Graham Neubig, Ryohei Sasano, Hiroya Takamura, and Manabu Okumura. Controlling output length in neural encoder-decoders. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1328–1338, 2016.
- [Klein et al., 2017] Guillaume Klein, Yoon Kim, Yuntian Deng, Jean Senellart, and Alexander M Rush. Opennmt: Open-source toolkit for neural machine translation. In *Proceedings of ACL 2017, System Demonstrations*, pages 67–72, 2017.
- [Knight, 1999] Kevin Knight. Decoding complexity in word-replacement translation models. *Computational linguistics*, 25(4):607–615, 1999.
- [Koehn, 2004] Philipp Koehn. Pharaoh: a beam search decoder for phrase-based statistical machine translation models. In *Conference of the Association for Machine Translation in the Americas*, pages 115–124. Springer, 2004.
- [Koehn, 2010] Philipp Koehn. *Statistical Machine Translation*. Cambridge University Press, 2010.
- [Koehn and Knowles, 2017] Philipp Koehn and Rebecca Knowles. Six challenges for neural machine translation. In *Proceedings of the First Workshop on Neural Machine Translation*, pages 28–39, 2017.
- [Koehn et al., 2003] Philipp Koehn, Franz Josef Och, and Daniel Marcu. Statistical phrase-based translation. In *Proceedings of the 2003 Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics*, pages 127–133, 2003.

- [Koehn et al., 2007] Philipp Koehn, Hieu Hoang, Alexandra Birch, Chris Callison-Burch, Marcello Federico, Nicola Bertoldi, Brooke Cowan, Wade Shen, Christine Moran, Richard Zens, Chris Dyer, Ondřej Bojar, Alexandra Constantin, and Evan Herbst. Moses: Open source toolkit for statistical machine translation. In *Proceedings of the 45th Annual Meeting of the Association for Computational Linguistics Companion Volume Proceedings of the Demo and Poster Sessions*, pages 177–180, 2007.
- [Kulikov et al., 2019] Ilia Kulikov, Alexander Miller, Kyunghyun Cho, and Jason Weston. Importance of search and evaluation strategies in neural dialogue modeling. In *Proceedings of the 12th International Conference on Natural Language Generation*, pages 76–87, 2019.
- [Kumar et al., 2016] Ankit Kumar, Ozan Irsoy, Peter Ondruska, Mohit Iyyer, James Bradbury, Ishaan Gulrajani, Victor Zhong, Romain Paulus, and Richard Socher. Ask me anything: Dynamic memory networks for natural language processing. In *International conference on machine learning*, pages 1378–1387, 2016.
- [Kumar et al., 2021] Sachin Kumar, Eric Malmi, Aliaksei Severyn, and Yulia Tsvetkov. Controlled text generation as continuous optimization with multiple constraints. *Advances in Neural Information Processing Systems*, 34:14542–14554, 2021.
- [Kumar and Byrne, 2004] Shankar Kumar and William Byrne. Minimum bayes-risk decoding for statistical machine translation. In *Proceedings of the Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics: HLT-NAACL 2004*, pages 169–176, 2004.
- [Lample and Conneau, 2019] Guillaume Lample and Alexis Conneau. Cross-lingual language model pretraining. *arXiv preprint arXiv:1901.07291*, 2019.
- [Leblond et al., 2021] Rémi Leblond, Jean-Baptiste Alayrac, Laurent Sifre, Miruna Pislari, Lespiau Jean-Baptiste, Ioannis Antonoglou, Karen Simonyan, and Oriol Vinyals. Machine translation decoding beyond beam search. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 8410–8434, 2021.
- [Lee et al., 2020] Jason Lee, Elman Mansimov, and Kyunghyun Cho. Deterministic non-autoregressive neural sequence modeling by iterative refinement. In *2018 Conference on Empirical Methods in Natural Language Processing, EMNLP 2018*, pages 1173–1182, 2020.
- [Lee et al., 2019] John Boaz Lee, Ryan A Rossi, Sungchul Kim, Nesreen K Ahmed, and Eunye Koh. Attention models in graphs: A survey. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 13(6):1–25, 2019.
- [Li et al., 2021] Jicheng Li, Pengzhi Gao, Xuanfu Wu, Yang Feng, Zhongjun He, Hua Wu, and Haifeng Wang. Mixup decoding for diverse machine translation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 312–320, 2021.
- [Li and Jurafsky, 2016] Jiwei Li and Dan Jurafsky. Mutual information and diverse decoding improve neural machine translation. *arXiv preprint arXiv:1601.00372*, 2016.
- [Li et al., 2016] Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and William B Dolan. A diversity-promoting objective function for neural conversation models. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 110–119, 2016.
- [Li et al., 2017] Jiwei Li, Will Monroe, and Dan Jurafsky. Learning to decode for future success. *arXiv preprint arXiv:1701.06549*, 2017.

- [Li et al., 2019] Xintong Li, Guanlin Li, Lemao Liu, Max Meng, and Shuming Shi. On the word alignment from neural machine translation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1293–1303, 2019.
- [Li et al., 2018] Yanyang Li, Tong Xiao, Yinqiao Li, Qiang Wang, Changming Xu, and Jingbo Zhu. A simple and effective approach to coverage-aware neural machine translation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 292–297, 2018.
- [Liu et al., 2016] Lemao Liu, Masao Utiyama, Andrew Finch, and Eiichiro Sumita. Agreement on target-bidirectional neural machine translation. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 411–416, 2016a.
- [Liu et al., 2016] Lemao Liu, Masao Utiyama, Andrew Finch, and Eiichiro Sumita. Neural machine translation with supervised attention. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 3093–3102, 2016b.
- [Lu et al., 2016] Jiasen Lu, Jianwei Yang, Dhruv Batra, and Devi Parikh. Hierarchical question-image co-attention for visual question answering. *Advances in neural information processing systems*, 29, 2016.
- [Luong et al., 2015] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1412–1421, 2015.
- [Ma et al., 2019] Mingbo Ma, Liang Huang, Hao Xiong, Renjie Zheng, Kaibo Liu, Baigong Zheng, Chuanqiang Zhang, Zhongjun He, Hairong Liu, Xing Li, Hua Wu, and Haifeng Wang. Stacl: Simultaneous translation with implicit anticipation and controllable latency using prefix-to-prefix framework. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3025–3036, 2019.
- [Malaviya et al., 2018] Chaitanya Malaviya, Pedro Ferreira, and André FT Martins. Sparse and constrained attention for neural machine translation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 370–376, 2018.
- [Matusov et al., 2006] Evgeny Matusov, Nicola Ueffing, and Hermann Ney. Computing consensus translation for multiple machine translation systems using enhanced hypothesis alignment. In *11th Conference of the European Chapter of the Association for Computational Linguistics*, pages 33–40, 2006.
- [Meister et al., 2020] Clara Meister, Tim Vieira, and Ryan Cotterell. Best-first beam search. *Transactions of the Association for Computational Linguistics*, 8:795–809, 2020.
- [Mi et al., 2016] Haitao Mi, Baskaran Sankaran, Zhiguo Wang, and Abe Ittycheriah. Coverage embedding models for neural machine translation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 955–960, 2016a.
- [Mi et al., 2016] Haitao Mi, Zhiguo Wang, and Abe Ittycheriah. Supervised attentions for neural machine translation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2283–2288, 2016b.
- [Müller and Sennrich, 2021] Mathias Müller and Rico Sennrich. Understanding the properties of minimum bayes risk decoding in neural machine translation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference*

- on Natural Language Processing (Volume 1: Long Papers)*, pages 259–272, 2021.
- [Murray and Chiang, 2018] Kenton Murray and David Chiang. Correcting length bias in neural machine translation. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 212–223, 2018.
- [Neisser, 2014] Ulric Neisser. *Cognitive Psychology: Classic Edition*. Psychology Press, 2014.
- [Nguyen et al., 2020] Xuan-Phi Nguyen, Shafiq Joty, Steven Hoi, and Richard Socher. Tree-structured attention with hierarchical accumulation. In *Proceedings of the 8th International Conference on Learning Representations ICLR*, 2020.
- [Och, 2003] Franz Josef Och. Minimum error rate training in statistical machine translation. In *Proceedings of the 41st annual meeting of the Association for Computational Linguistics*, pages 160–167, 2003.
- [Och and Ney, 2002] Franz Josef Och and Hermann Ney. Discriminative training and maximum entropy models for statistical machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 295–302, 2002.
- [Och and Ney, 2003] Franz Josef Och and Hermann Ney. A systematic comparison of various statistical alignment models. *Computational Linguistics*, 29(1):19–51, 2003.
- [Opitz and Maclin, 1999] David Opitz and Richard Maclin. Popular ensemble methods: An empirical study. *Journal of artificial intelligence research*, 11:169–198, 1999.
- [Ott et al., 2018] Myle Ott, Michael Auli, David Grangier, and Marc’Aurelio Ranzato. Analyzing uncertainty in neural machine translation. In *International Conference on Machine Learning*, pages 3956–3965. PMLR, 2018a.
- [Ott et al., 2018] Myle Ott, Sergey Edunov, David Grangier, and Michael Auli. Scaling neural machine translation. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 1–9, October 2018b.
- [Ott et al., 2019] Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli. fairseq: A fast, extensible toolkit for sequence modeling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, pages 48–53, 2019.
- [Post and Vilar, 2018] Matt Post and David Vilar. Fast lexically constrained decoding with dynamic beam allocation for neural machine translation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1314–1324, 2018.
- [Raffel et al., 2017] Colin Raffel, Minh-Thang Luong, Peter J Liu, Ron J Weiss, and Douglas Eck. Online and linear-time attention by enforcing monotonic alignments. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 2837–2846, 2017.
- [Raffel et al., 2020] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- [Ren et al., 2017] Zhou Ren, Xiaoyu Wang, Ning Zhang, Xutao Lv, and Li-Jia Li. Deep reinforcement learning-based image captioning with embedding reward. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 290–298, 2017.
- [Rosti et al., 2007] Antti-Veikko Rosti, Spyros Matsoukas, and Richard Schwartz. Improved word-level

- system combination for machine translation. In *Proceedings of the 45th Annual Meeting of the Association of Computational Linguistics*, pages 312–319, 2007.
- [Ruder, 2017] Sebastian Ruder. Deep learning for nlp best practices. <https://ruder.io/deep-learning-nlp-best-practices/index.html>, 2017.
- [Rush and Collins, 2012] Alexander M Rush and MJ Collins. A tutorial on dual decomposition and lagrangian relaxation for inference in natural language processing. *Journal of Artificial Intelligence Research*, 45:305–362, 2012.
- [Rush et al., 2015] Alexander M Rush, Sumit Chopra, and Jason Weston. A neural attention model for abstractive sentence summarization. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 379–389, 2015.
- [Russell and Norvig, 2010] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach (3rd ed.)*. Prentice Hall, 2010.
- [Sankaran et al., 2016] Baskaran Sankaran, Haitao Mi, Yaser Al-Onaizan, and Abe Ittycheriah. Temporal attention model for neural machine translation. *arXiv preprint arXiv:1608.02927*, 2016.
- [See, 2018] Abigail See. Deep learning, structure and innate priors: A discussion between yann lecun and christopher manning, 02 2018. URL <http://www.abigailsee.com/2018/02/21/deep-learning-structure-and-innate-priors.html>.
- [See et al., 2017] Abigail See, Peter J Liu, and Christopher D Manning. Get to the point: Summarization with pointer-generator networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1073–1083, 2017.
- [Sennrich et al., 2016] Rico Sennrich, Barry Haddow, and Alexandra Birch. Improving neural machine translation models with monolingual data. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 86–96, 2016.
- [Shannon, 1948] Claude E. Shannon. A mathematical theory of communication. Report, Bell Labs, 1948.
- [Shen et al., 2016] Shiqi Shen, Yong Cheng, Zhongjun He, Wei He, Hua Wu, Maosong Sun, and Yang Liu. Minimum risk training for neural machine translation. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1683–1692, 2016.
- [Shen et al., 2019] Tianxiao Shen, Myle Ott, Michael Auli, and Marc’Aurelio Ranzato. Mixture models for diverse machine translation: Tricks of the trade. In *International conference on machine learning*, pages 5719–5728, 2019.
- [Silver et al., 2017] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.
- [Stahlberg and Byrne, 2019] Felix Stahlberg and Bill Byrne. On nmt search errors and model errors: Cat got your tongue? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3356–3362, 2019.
- [Stahlberg et al., 2016] Felix Stahlberg, Eva Hasler, Aurelien Waite, and Bill Byrne. Syntactically guided neural machine translation. In *Proceedings of the 54th Annual Meeting of the Association for*

- Computational Linguistics (Volume 2: Short Papers)*, pages 299–305, 2016.
- [Sternberg, 1996] Robert J Sternberg. *Cognitive psychology*. Harcourt Brace College Publishers, 1996.
- [Sukhbaatar et al., 2015] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. *Advances in neural information processing systems*, 28, 2015.
- [Sun et al., 2020] Zewei Sun, Shujian Huang, Hao-Ran Wei, Xin-yu Dai, and Jiajun Chen. Generating diverse translation by manipulating multi-head attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8976–8983, 2020.
- [Sutskever et al., 2014] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, 27, 2014.
- [Taskar et al., 2005] Ben Taskar, Simon Lacoste-Julien, and Dan Klein. A discriminative matching approach to word alignment. In *Proceedings of Human Language Technology Conference and Conference on Empirical Methods in Natural Language Processing*, pages 73–80, 2005.
- [Tay et al., 2020] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *CoRR*, abs/2009.06732, 2020.
- [Tu et al., 2016] Zhaopeng Tu, Zhengdong Lu, Yang Liu, Xiaohua Liu, and Hang Li. Modeling coverage for neural machine translation. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 76–85, 2016.
- [Vaswani et al., 2017] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of Advances in Neural Information Processing Systems*, volume 30, 2017.
- [Veličković et al., 2018] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- [Vijayakumar et al., 2018] Ashwin Vijayakumar, Michael Cogswell, Ramprasaath Selvaraju, Qing Sun, Stefan Lee, David Crandall, and Dhruv Batra. Diverse beam search for improved description of complex scenes. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- [Vogel et al., 1996] Stephan Vogel, Hermann Ney, and Christoph Tillmann. Hmm-based word alignment in statistical translation. In *COLING 1996 Volume 2: The 16th International Conference on Computational Linguistics*, 1996.
- [Wiher et al., 2022] Gian Wiher, Clara Meister, and Ryan Cotterell. On decoding strategies for neural text generators. *Transactions of the Association for Computational Linguistics*, 10:997–1012, 2022.
- [Wu et al., 2020] Xuanfu Wu, Yang Feng, and Chenze Shao. Generating diverse translation from model distribution with dropout. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1088–1097, 2020.
- [Wu et al., 2016] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- [Xiao et al., 2013] Tong Xiao, Jingbo Zhu, and Tongran Liu. Bagging and boosting statistical machine

- translation systems. *Artificial Intelligence*, 195:496–527, 2013.
- [Xu et al., 2015] Kelvin Xu, Jimmy Ba, Ryan Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhudinov, Rich Zemel, and Yoshua Bengio. Show, attend and tell: Neural image caption generation with visual attention. In *International conference on machine learning*, pages 2048–2057. PMLR, 2015.
- [Xu et al., 2021] Zenan Xu, Daya Guo, Duyu Tang, Qinliang Su, Linjun Shou, Ming Gong, Wanjun Zhong, Xiaojun Quan, Daxin Jiang, and Nan Duan. Syntax-enhanced pre-trained model. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 5412–5422, 2021.
- [Yang et al., 2018] Baosong Yang, Zhaopeng Tu, Derek F Wong, Fandong Meng, Lidia S Chao, and Tong Zhang. Modeling localness for self-attention networks. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 4449–4458, 2018a.
- [Yang et al., 2018] Yilin Yang, Liang Huang, and Mingbo Ma. Breaking the beam search curse: A study of (re-) scoring methods and stopping criteria for neural machine translation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3054–3059, 2018b.
- [You et al., 2020] Weiqiu You, Simeng Sun, and Mohit Iyyer. Hard-coded gaussian attention for neural machine translation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7689–7700, 2020.
- [Zaslavskiy et al., 2009] Mikhail Zaslavskiy, Marc Dymetman, and Nicola Cancedda. Phrase-based statistical machine translation as a traveling salesman problem. In *Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP*, pages 333–341, 2009.
- [Zhang et al., 2020] Jiajun Zhang, Long Zhou, Yang Zhao, and Chengqing Zong. Synchronous bidirectional inference for neural sequence generation. *Artificial Intelligence*, 281:103234, 2020a.
- [Zhang et al., 2018] Xiangwen Zhang, Jinsong Su, Yue Qin, Yang Liu, Rongrong Ji, and Hongji Wang. Asynchronous bidirectional decoding for neural machine translation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [Zhang et al., 2020] Zhuosheng Zhang, Yuwei Wu, Junru Zhou, Sufeng Duan, Hai Zhao, and Rui Wang. Sg-net: Syntax-guided machine reading comprehension. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 9636–9643, 2020b.
- [Zheng et al., 2019] Baigong Zheng, Renjie Zheng, Mingbo Ma, and Liang Huang. Simpler and faster learning of adaptive policies for simultaneous translation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1349–1354, 2019.
- [Zheng et al., 2018] Zaixiang Zheng, Hao Zhou, Shujian Huang, Lili Mou, Xinyu Dai, Jiajun Chen, and Zhaopeng Tu. Modeling past and future for neural machine translation. *Transactions of the Association for Computational Linguistics*, 6:145–157, 2018.
- [Zhou et al., 2017] Long Zhou, Wenpeng Hu, Jiajun Zhang, and Chengqing Zong. Neural system combination for machine translation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 378–384, 2017.

[Zhou, 2012] Zhi-Hua Zhou. *Ensemble methods: foundations and algorithms*. CRC press, 2012.